

Contents

1	Programsko inženjerstvo ak.god 2025./2026.	4
1.1	Sveučilište u Zagrebu	4
1.2	Fakultet elektrotehnike i računarstva	4
2	Dvoranko	4
3	Opis projektnog zadatka	4
4	Cilj projektnog zadatka	5
5	Problematika projektnog zadatka	5
6	Postojeća slična rješenja	5
7	Opseg projektnog zadatka	7
8	Moguće nadogradnje	8
9	Funkcionalni zahtjevi	8
10	Nefunkcionalni zahtjevi	10
10.1	Zahtjevi za performanse	10
10.2	Zahtjevi za pouzdanost i dostupnost	10
10.3	Zahtjevi za upotrebljivost	10
10.4	Zahtjevi za kompatibilnost	10
10.5	Zahtjevi za sigurnost	10
11	Dionici	11
11.1	Aktori i njihovi funkcionalni zahtjevi	11
12	Obrasci uporabe	12
13	Dijagrami obrazaca uporabe	12
13.1	1. Visokorazinski dijagram obrazaca uporabe cijelog sustava	12
13.2	2. dijagram obrazaca uporabe za ključne značajke	13
13.3	3. dijagram obrazaca uporabe za korisničke uloge	14
13.3.1	3.1 Neprijavljeni korisnik	14
13.3.2	3.2 Prijavljeni korisnik	14
13.3.3	3.3 Vlasnik dvorane (iznajmljivač)	15
13.3.4	3.4 Administrator	15
13.3.5	3.5 Sustav e-pošte (automatizirani akter)	15
14	Opis obrazaca uporabe	15
14.1	UC1 – Registracija korisnika	15
14.2	UC2 – Prijava korisnika	16
14.3	UC3 – Pregled dostupnih dvorana	16
14.4	UC4 – Pregled slobodnih termina	16
14.5	UC5 – Rezervacija termina	16
14.6	UC6 – Pregled javnih događanja	17
14.7	UC7 – Pregled vlastitih rezervacija	17
14.8	UC8 – Zahtjev za promjenom uloge	17
14.9	UC9 – Slanje zahtjeva za objavu dvorane	17
14.10	UC10 – Upravljanje vlastitim dvoranama	18
14.11	UC11 – Pregled zahtjeva za rezervacije	18

14.12UC12 – Prihvaćanje / odbijanje rezervacija	18
14.13UC13 – Primanje e-mail obavijesti	18
14.14UC14 – Pregled zahtjeva za ulogu vlasnika	18
14.15UC15 – Pregled zahtjeva za objavu dvorana	19
14.16UC16 – Nadzor nad korisnicima i dvoranama	19
14.17UC17 – Obavješćavanje korisnika putem e-maila	19
15 Sekvencijski dijagrami	20
16 Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe	21
17 Arhitektura sustava	23
17.1 Opis arhitekture	23
17.1.1 Stil arhitekture	23
17.1.2 Podsustavi	23
17.1.3 Preslikavanje na radnu platformu	23
17.1.4 Spremišta podataka	23
17.1.5 Mrežni protokoli	23
17.1.6 Globalni upravljački tok	23
17.1.7 Sklopovskoprogramski zahtjevi	23
17.2 Obrazloženje odabira arhitekture	24
17.2.1 Izbor arhitekture temeljen na principima oblikovanja	24
17.2.2 Razmatrane alternative	24
17.3 Organizacija sustava na visokoj razini	24
17.3.1 Klijent-poslužitelj	24
17.3.2 Baza podataka	24
17.3.3 Grafičko sučelje	24
17.4 Organizacija aplikacije	28
17.4.1 Frontend i Backend slojevi	28
18 Baza podataka	28
18.1 Opis tablica	28
18.1.1 Users	28
18.1.2 Dvorana	28
18.1.3 Dvorana_kategorija	29
18.1.4 Kategorija	29
18.1.5 Termin	30
18.1.6 Adresa	30
18.1.7 Mjesto	31
18.1.8 Zahtjev_iznajmljivac	31
18.1.9 Zahtjev_oglas	31
18.1.10 Zahtjev_oglas_kategorija	32
18.1.11 Zahtjev_slika	33
18.1.12 Zahtjev_termin	33
18.1.13 Slika_dvorana	34
18.2 Dijagram baze podataka	34
19 Dijagram razreda	36
19.1 Cijela Web-Aplikacija	36
19.2 Modeli	38
19.3 Kontroleri	39
19.4 DTO	40
19.5 Servisi	41
19.6 Sigurnost i pomoćne klase	42

20 Dinamičko ponašanje aplikacije	43
20.1 UML dijagrami stanja	43
20.2 UML dijagrami aktivnosti	45
20.3 Dijagram komponenata	46
20.4 Dijagram razmještaja	47
21 Ispitivanje sustava	55
21.1 1. Ispitni slučaj: Provjera uspješnosti ažuriranja podataka dvorane	55
21.2 2. Ispitni slučaj: Provjera uspješnosti ažuriranja podataka dvorane	55
21.3 3. Ispitni slučaj: Provjera uspješnosti kreiranja rezervacije	58
21.4 4. Ispitni slučaj: Provjera uspješnosti filtriranja dvorana	61
22 Korištene tehnologije i alati	63
22.1 Programski jezici	63
22.2 Radni okviri i biblioteke	63
22.3 Baza podataka	63
22.4 Razvojni alati	64
22.5 Alati za ispitivanje	64
22.6 Alati za razmještaj	64
22.7 Cloud platforma	64
23 1. Instalacija	64
24 2. Postavke	65
24.1 Detaljne upute za konfiguraciju aplikacije:	65
24.1.1 Konfiguracijske datoteke:	65
24.1.2 Postavke baze podataka:	65
25 3. Pokretanje aplikacije	66
25.0.1 Upute za pokretanje aplikacije u različitim okruženjima:	66
26 4. Upute za administratore	66
26.0.1 Smjernice za administratore aplikacije nakon puštanja u pogon:	66
27 5. Postavljanje na platformu Render	67
28 6. Opis pristupa aplikaciji na javnom poslužitelju	68
29 Dnevnik sastajanja	69
30 Plan rada	70
31 Tablica aktivnosti	70
32 Dijagram pregleda promjena	71
33 Kjučni izazovi i rješenja	71

1 Programsko inženjerstvo ak.god 2025./2026.

1.1 Sveučilište u Zagrebu

1.2 Fakultet elektrotehnike i računarstva

2 Dvoranko

2.0.0.0.1 Tim: TG10.1

2.0.0.0.2 Ime tima : LeProgi

2.0.0.0.3 Članovi: Vid Martin Lulić, Dino Maras, Borna Navratil, Eva Huljaj, Matija Vandek, Ivan Vučak

2.0.0.0.4 Nastavnik: Vlado Sruk

3 Opis projektnog zadatka

U urbanim sredinama poput Zagreba, sve je veća potreba za pronalaženjem i rezervacijom prostora za različite namjene: radionice, konferencije, proslave ili sportske aktivnosti. Trenutačno korisnici najčešće pretražuju i kontaktiraju dvorane ručno putem interneta, društvenih mreža, telefona ili e-maila, što je dugotrajan i često neorganiziran proces. Upravo iz te potrebe proizašla je naša ideja Dvoranko web-aplikacije, čiji je cilj omogućiti jednostavniju, bržu i pregledniju rezervaciju dvorana u Zagrebu na jednom mjestu. Dvoranko okuplja ponudu različitih prostora u jedinstven sustav, čime olakšava komunikaciju između iznajmljivača i korisnika te modernizira proces rezervacije i oglašavanja dvorana.

Aplikaciju Dvoranko moguće je koristiti kao običan korisnik, vlasnik dvorane ili administrator. Kada korisnik prvotno otvori aplikaciju nazivamo ga neprijavljeni korisnik. On tada ima pristup početnoj stranici aplikacije, gdje mu se prikazuju objavljene dvorane, i oglasnoj ploči te se mora prijaviti kako bi postao običan (prijavljeni) korisnik. Obični korisnici imaju mogućnost pretraživanja i rezervacije dvorana. Tijekom rezervacije korisnik mora definirati namjenu prostora ako je iznajmljivač to zatražio, broj gostiju te označiti hoće li događanje biti javno ili privatno. U slučaju javnog događanja, ono postaje vidljivo svim korisnicima unutar aplikacije, čime se potiče sudjelovanje zajednice i promocija događanja u gradu. Prijavljenim korisnicima je omogućeno i upravljanje vlastitim rezervacijama. Mogu ih pregledavati, mijenjati ili otkazati prema pravilima sustava.

Vlasnici dvorana imaju dodatne funkcionalnosti koje im omogućuju upravljanje svojim prostorima. Nakon registracije, obični korisnik može zatražiti nadogradnju uloge u vlasnika dvorane. Kada administrator odobri zahtjev, vlasnik može dodavati nove dvorane, unositi njihove opise, veličine, kapacitete i fotografije te određivati cijene i dostupne termine. Sustav automatski obavještava vlasnika putem e-maila kada korisnik pošalje zahtjev za rezervaciju termina. Vlasnik može pregledati sve zahtjeve, prihvatiti ih ili odbiti, te tako jednostavno upravljati svojim poslovanjem putem aplikacije.

Administrator ima najvišu razinu pristupa i odgovoran je za održavanje sigurnosti unutar sustava. Njegove funkcionalnosti uključuju odobravanje zahtjeva korisnika za objavu dvorana te potvrdu promjene korisničke uloge u ulogu iznajmljivača dvorane ili prostora. Također, administrator prati rad aplikacije, provjerava točnost unesenih podataka te ima uvid u sve rezervacije i korisničke aktivnosti kako bi osigurali pouzdanost i transparentnost sustava.

Svi korisnici aplikaciji pristupaju putem prijave koristeći e-mail adresu, a autentifikacija se odvija uz pomoć OAuth2 protokola radi zaštite osobnih podataka. Aplikacija je u potpunosti responzivna i prilagođena korištenju na različitim uređajima, računalima i mobilnim telefonima, čime se osigurava jednostavan pristup i preglednost.

Kroz ovakav sustav, Dvoranko omogućuje digitalizaciju i unaprjeđenje procesa rezervacije prostora u Zagrebu. Korisnici mogu pronaći dvoranu koja im odgovara u svega nekoliko klikova, dok vlasnici mogu učinkovito

upravljati svojim prostorima i terminima bez potrebe za dodatnim alatima. Administratori osiguravaju da cijeli sustav funkcionira pouzdano i sigurno. Na taj način, Dvoranko stvara povezan “ekosustav” u kojem su svi sudionici uključeni u transparentan i učinkovit proces rezervacije dvorana.

4 Cilj projektnog zadatka

Osnovni cilj ovog projektnog zadatka je omogućiti korisnicima jednostavno pretraživanje i rezervaciju prostora za različite svrhe poput sastanaka, radionica, sportskih aktivnosti i sajмова, a vlasnicima prostora objavljivanje i iznajmljivanje svojih prostora. Dvoranko omogućuje centralizirano mjesto za sve ponude dvorana u Zagrebu. Ideja je napraviti pristupačno sučelje za sve korisnike kako bi se prije navedeni proces olakšao i modernizirao.

Dvoranko web-aplikacija pruža korisnicima brzo i efikasno rezerviranje termina i organiziranje javnih ili privatnih događanja. S druge strane pruža vlasnicima prostora mogućnost oglašavanja svojih prostora bez potrebe za izradom vlastitih stranica i sustava oglašavanja. Konačni cilj je stvoriti centraliziranu platformu koja će objediniti ponudu javnih i privatnih dvorana u Zagrebu, olakšati korisnicima pronalazak slobodnih termina te povećati iskorištenost dostupnih prostora.

5 Problematika projektnog zadatka

Trenutačno u Zagrebu ne postoji jedinstveno digitalno rješenje koje omogućuje jednostavno pretraživanje i rezervaciju dvorana ili prostora. Dosadašnji načini rezervacija dvorana iznimno ovise o tome u čijem su vlasništvu i koliko se te osobe samostalno promoviraju. Najčešći način rezerviranja ovakvih prostora je putem mobilnih telefona ili putem e-maila. Iako su ove metode jednostavne i poznate većini ljudi, takvim pristupom se cjelokupni proces znatno usporava. Korisnici koji traže prostor često moraju kontaktirati više lokacija zasebno što troši i njihovo vrijeme, a i vrijeme voditelja prostora koji moraju odgovarati na upite, te povećava vjerojatnost pogreške pri rezervacijama. Naša aplikacija nudi način ubrzanja ovog procesa gdje bi korisnici na jednom mjestu imali pregled slobodnih termina dvorana, čiji su ih vlasnici oglasili u sustavu.

Još jedan od problema na koji bi naša aplikacija utjecala je promoviranje javnih događanja. Brojni sajmovi ili predstave se svakodnevno održavaju u Zagrebu za koje veliki broj ljudi ne zna. Ako korisnik rezervira prostor u svrhe javnog događanja, može rezervaciju označiti upravo kao javno događanje te ga oglasiti na oglasnoj ploči aplikacije. Ostali korisnici će imati pregled javnih događanja koja se odvijaju u gradu te će se moći prijaviti na ono što ih zanima.

6 Postojeća slična rješenja

Postoji mnogo aplikacija i web stranica koje nude rješenja za ovaj problem. Većina tih rješenja se fokusira na rezervaciju termina za sportska događanja poput turnira i treninga. U tom aspektu se naša aplikacija razlikuje od već postojećih.

Jedan od primjera sličnog rješenja je Sport Router. Sport Router je mobilna aplikacija koja korisnicima pruža intuitivno i jednostavno iskustvo rezerviranja sportskih terena za vlastite potrebe. Termine je moguće pretraživati po vrsti sporta, željenom vremenu te lokaciji terena ili dvorane. Kada korisnik instalira aplikaciju može isključivo rezervirati termin. Za objavu vlastitog centra potrebno je ispuniti prijavu na njihovoj web stranici (<https://sportrouter.com/#connectYourCenter>).

Korisnicima se nakon prijave otvara jednostavno korisničko sučelje s izbornikom sportova i izbornikom datuma u tekućem tjednu. Kada korisnik odabere sport i datum, ispod toga mu se prikazuju slobodni termini na različitim lokacijama (slika 1.1). Na slici je prikazano korisničko sučelje za mobilne aplikacije.

Odabirom određene dvorane vidljivi su podaci o samom prostoru. Vidljiva je lokacija prostora, radno vrijeme te opis prostora koji vlasnik napiše kada objavljuje svoj prostor u aplikaciji (slika 1.2). Vidljivi su termini po danima u tjednu te raspoređeni po satima.

Naša aplikacija ima nekoliko sličnih funkcionalnosti i uporaba kao i Sport Router. Neke od njih su: mogućnost prijave kao korisnik i promjene te uloge u iznajmljivača, mogućnost opisivanja prostora koji korisnik objavljuje,



Figure 1: Slika 1

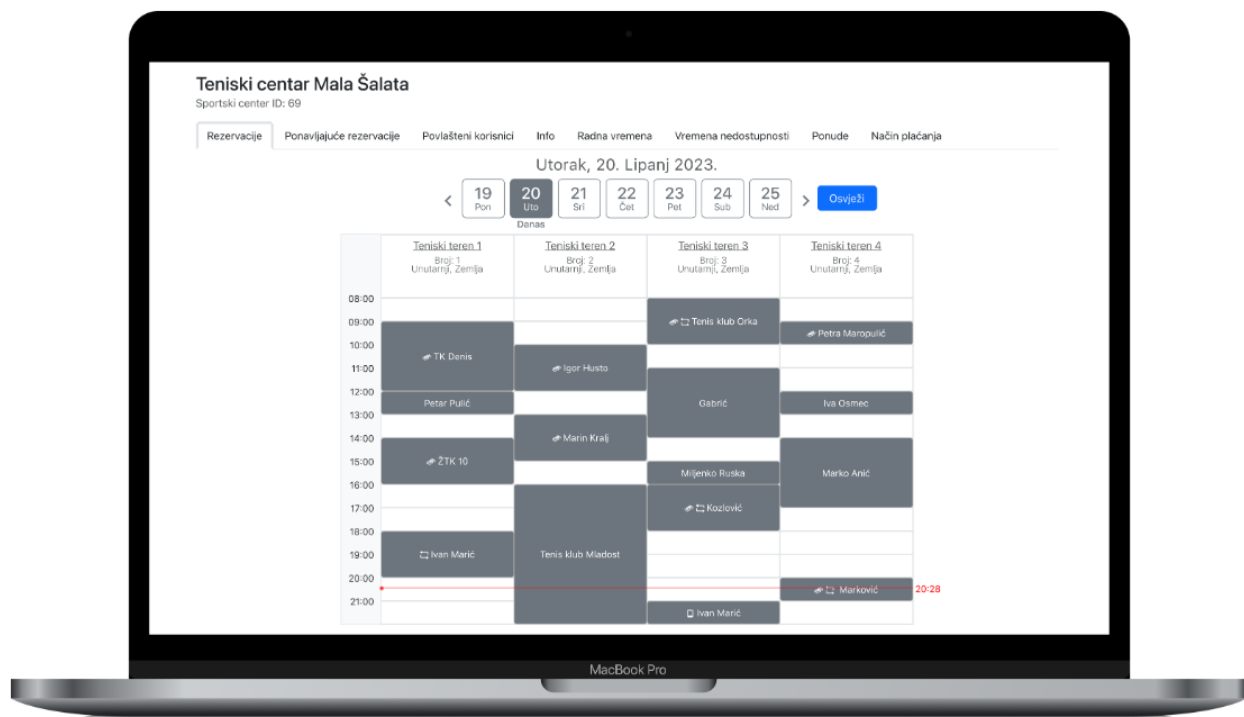


Figure 2: Slika 1

pregled objavljenih prostora. Razliku pronalazimo u namjeni prostora ili razlogu rezervacije. Dvoranko omogućuje rezervaciju dvorana ili prostora za javna događanja poput sajмова, predstava uz mogućnost rezervacije dvorana za sportske aktivnosti.

7 Opseg projektnog zadatka

Projekt Dvoranko obuhvaća izradu web-aplikacije koja omogućuje centralizirano i jednostavno upravljanje procesom pretraživanja, rezervacije i oglašavanja dvorana u Zagrebu. Sustav je zamišljen kao trodijelna platforma koja povezuje korisnike, vlasnike dvorana i administratore u jedinstveno digitalno okruženje.

Funkcionalni opseg

1. Neprijavljeni korisnici > - Pregled javno dostupnih dvorana i oglasne ploče. > - Pregled prikaza dvorana na karti > - Registracija i prijava u sustav.
2. Prijavljeni korisnici (obični korisnici) > - Detaljno pretraživanje i filtriranje dostupnih dvorana prema različitim parametrima (cijena, veličina, dostupnost termina, namjena prostora). > - Pregled informacija o dvoranama (opis, fotografije, dostupni termini). > - Rezervacija dvorane uz unos potrebnih informacija (broj gostiju, namjena, javno/privatno događanje). > - Upravljanje vlastitim rezervacijama – pregled, izmjena i otkazivanje. > - Zahtjev za promjenu korisničke uloge iz običnog korisnika u vlasnika dvorane.
3. Vlasnici dvorana > - Unos i upravljanje vlastitim prostorima (dodavanje, uređivanje i brisanje dvorana). > - Definiranje detalja o prostoru (opis, kapacitet, veličina, cijena, fotografije, dostupni termini). > - Pregled zaprimljenih zahtjeva za rezervacije i upravljanje njima (odobranje ili odbijanje zahtjeva). > - Primanje automatskih obavijesti e-mailom o novim zahtjevima za rezervaciju.
4. Administratori > - Odobranje zahtjeva korisnika za promjenu uloge u vlasnika dvorane. > - Pregled i verifikacija unesenih podataka o dvoranama. > - Upravljanje oglasima i održavanje integriteta sustava.

> - Uvid u sve rezervacije i korisničke aktivnosti radi kontrole i sigurnosti sustava.

Tehnički opseg > - Platforma: Web-aplikacija dostupna putem internetskog preglednika. > - Autentifikacija: Prijava putem e-mail adrese uz korištenje OAuth2 protokola radi sigurnosti korisničkih podataka. > - Responzivni dizajn: Prilagođen prikaz i funkcionalnosti za rad na računalima, tabletima i mobilnim uređajima. > - Baza podataka: Centralizirano spremište podataka o korisnicima, dvoranama, rezervacijama i javnim događanjima. > - E-mail notifikacije: Automatsko slanje obavijesti korisnicima i vlasnicima o relevantnim aktivnostima u sustavu (potvrda registracije, zahtjevi za rezervaciju, odobrenja uloga itd.).

8 Moguće nadogradnje

Dvoranko je osmišljen kao fleksibilna platforma koja je lako prilagodljiva različitim potrebama korisnika. Naša trenutačna aplikacija se koncentrira na problem rezervacija dvorana u Zagrebu. Jedna od mogućih prilagodba je proširiti područje na druge gradove poput Rijeke, Splita ili Osijeka. Moguće se proširiti i na druge vrste prostora poput kazališta, koncertnih dvorana, konferencijskih sala i slično. Omogućili bismo rezervaciju kazališta za predstave ili konferencijskih sali za domjenke čime bi se baza klijenata još više povećala.

Druga potencijalna nadogradnja je izrada mobilne aplikacije za Android ili iOS platforme, što bi korisnicima omogućilo brži i pristupačniji način rezervacije dvorana. Mobilna verzija mogla bi sadržavati push notifikacije za nove ponude dvorana ili novo dostupne termine. Daljnja nadogradnja može uključivati dodatne funkcionalnosti. Može se integrirati sustav ocjenjivanja prostora koji bi korisnicima omogućio davanje povratne informacije o prostoru ili usluzi. Time bi se povećala transparentnost među korisnicima te bi vlasnici dvorana ili prostora dobili uvid u kvalitetu prostora koje nude. Mogao bi se integrirati i prikaz statistike vlasnicima prostora koji uključuje broj novih klijenata, ukupan broj rezervacija i slično.

9 Funkcionalni zahtjevi

ID	Opis	Prioritet	Kriterij prihvaćanja
F-01	Sustav omogućuje korisnicima prijavu putem Google računa koristeći OAuth2 autentifikaciju.	Visok zahtjev dionika	Korisnik koji se prvi put prijavi putem Google računa automatski se registrira i pohranjuje u bazu podataka.
F-02	Sustav podržava tri korisničke uloge: obični korisnik, vlasnik dvorane i administrator.	Visok postojeći sustav	Novi korisnici dobivaju ulogu "običnog korisnika". Administrator i vlasnici dvorana imaju dodatne ovlasti definirane u sustavu.
F-03	Sustav omogućuje neprijavljenim korisnicima pregled javnog popisa dvorana (osnovni podaci: naziv, lokacija, kapacitet)	Visok zahtjev dionika	Neprijavljeni korisnici mogu pregledavati popis dvorana, ali ne mogu otvoriti detalje niti rezervirati dvoranu.
F-04	Sustav omogućuje neprijavljenim korisnicima pregled javnih događanja na oglasnoj ploči.	Visok zahtjev dionika	Neprijavljeni korisnici mogu pregledavati oglasnu ploču.
F-05	Sustav prikazuje sve objavljene dvorane na interaktivnoj karti s osnovnim informacijama dostupnim svim korisnicima.	Srednji zahtjev dionika	Na karti se prikazuju označene sve objavljene dvorane s osnovnim podacima.
F-06	Samo prijavljeni korisnici mogu pregledati detaljne informacije o dvorani (opis, slike, cijene, dostupni termini).	Visok zahtjev dionika	Pokušaj neprijavljenog korisnika da otvori detalje preusmjerava ga na stranicu za prijavu.

ID	Opis	Prioritet	Kriterij prihvaćanja
F-07	Sustav omogućuje prijavljenim korisnicima rezervaciju slobodnih termina određene dvorane.	Visokozhtjevdionika	Prijavljeni korisnik može odabrati slobodan termin i rezervirati ga.
F-08	Sustav prilikom rezervacije zahtijeva unos svrhe događanja, tipa događanja (javno/privatno) i broja gostiju.	Visokpostojećisustav	Sustav ne dopušta završetak rezervacije dok svi obavezni podatci nisu uneseni.
F-09	Sustav omogućuje korisniku pregled, izmjenu i otkazivanje vlastitih rezervacija.	Srednjidokumentzahtjeva	Korisnik na profilu može pregledati i upravljati vlastitim rezervacijama.
F-10	Korisnik može zatražiti promjenu uloge u vlasnika dvorane (iznajmljivača).	Visokdokumentzahtjeva	Korisnik ispunjava zahtjev za promjenu uloge koji administrator mora potvrditi ili odbiti.
F-11	Vlasnik dvorane može oglasiti svoju dvoranu putem aplikacije.	Visokdokumentzahtjeva	Vlasnik unosi podatke o dvorani (opis, slike, kapacitet, cijenu, lokaciju) i šalje zahtjev administratoru.
F-12	Svaka objavljena dvorana mora sadržavati sliku, opis i lokaciju.	Srednjipostojećisustav	Sustav ne dopušta objavu dvorane bez svih obaveznih informacija.
F-13	Svaki termin mora sadržavati datum, vrijeme i cijenu najma.	Visokpostojećisustav	Sustav ne dopušta objavu termina bez unesenih podataka o datumu, vremenu i cijeni.
F-14	Administrator može odobriti ili odbiti zahtjeve za objavu novih dvorana.	Visokpovratneinformacijekorisnika	Administrator odlučuje o zahtjevu, a sustav automatski šalje obavijest vlasniku dvorane.
F-15	Administrator može odobriti ili odbiti zahtjeve korisnika za promjenu uloge u vlasnika dvorane.	Visokpovratneinformacijekorisnika	Sustav obavještava korisnika o odluci administratora elektroničkom poštom.
F-16	Sustav automatski šalje e-mail obavijest korisniku o statusu zahtjeva za rezervaciju (odobreno/odbijeno).	Srednjipovratneinformacijekorisnika	Nakon odluke vlasnika, korisnik automatski prima obavijest putem e-pošte.
F-17	Sustav automatski šalje e-mail obavijest vlasniku dvorane o novom zahtjevu za rezervaciju.	Srednjipovratneinformacijekorisnika	Kada korisnik rezervira termin, vlasnik automatski prima e-mail s informacijama o zahtjevu.
F-18	Sustav omogućuje vlasniku uređivanje informacija o svojim objavljenim dvoranama.	Srednjipostojećisustav	Vlasnik može mijenjati opis, slike, cijenu i termine dvorane, a izmjene se spremaju u bazu.
F-19	Sustav omogućuje korisnicima filtriranje i pretraživanje dvorana po lokaciji, kapacitetu i cijeni.	Srednjizahtjevdionika	Korisnik može primijeniti filtere za pretragu dvorana prema zadanim parametrima.

F-20	Sustav omogućuje administratoru pregled svih korisnika, dvorana i rezervacija te njihovo uređivanje ili brisanje.	Visokdokument zahtjeva	Administrator ima pristup administracijskoj ploči i može mijenjati ili uklanjati zapise u sustavu.
------	---	---------------------------	--

10 Nefunkcionalni zahtjevi

10.1 Zahtjevi za performanse

ID zahtjeva	Opis	Prioritet
NF-1.1	Web-aplikacija mora u potpunosti učitati sve elemente glavnih stranica unutar 2-3 sekunde (Home page, login page).	Visok

10.2 Zahtjevi za pouzdanost i dostupnost

ID zahtjeva	Opis	Prioritet
NF-2.1	Aplikacija je dostupna 24 sata dnevno tokom cijele godine osim za vrijeme održavanja ("downtime")	Visok
NF-2.2	Sustav mora osigurati da podatci o prijavama ne budu izgubljeni ili oštećeni (sve uspješno obavljene rezervacije moraju biti trajno spremljene u bazi podataka)	Visok

10.3 Zahtjevi za upotrebljivost

ID zahtjeva	Opis	Prioritet
NF-3.1	Mogućnost povećavanja i smanjivanja fonta	Manji
NF-3.2	Aplikacija mora biti jednostavna za korištenje, s jasnim navigacijskim elementima za korisnike svih razina tehničke pismenosti.	Srednji

10.4 Zahtjevi za kompatibilnost

ID zahtjeva	Opis	Prioritet
NF-4.1	Aplikacija mora raditi ispravno na najnovijim verzijama svih glavnih preglednika (Chrome, Firefox, Safari, Edge).	Visok
NF-4.2	Sučelje aplikacije treba biti responzivnog dizajna te podržavati rad na mobilnim uređajima unutar web preglednika	Srednji

10.5 Zahtjevi za sigurnost

ID zahtjeva	Opis	Prioritet
NF-5.1	Sustav koristi OAuth2 za prijavu i upravljanje sesijama kako bi osigurao siguran pristup korisničkim podacima.	Visok
NF-5.2	Osobni podaci korisnika (e-mail, lozinke, povijest rezervacija,...) moraju biti pohranjeni u skladu s važećim propisima (GDPR).	Visok
NF-5.3	Pristup podacima i funkcionalnostima mora biti ograničen na temelju korisničkih uloga	Visok

11 Dionici

D1 -> neprijavljeni korisnik - krajnji korisnici koji imaju pristup početnoj stranici web-aplikacije, oglasnoj ploči i karti te se za daljnje korištenje moraju registrirati (ulogirati)

D2 -> prijavljeni korisnik - korisnici koji rezerviraju termine za željene dvorane, prijavljuju se na javna događanja te koriste ostale funkcionalnosti aplikacije

D3 -> iznajmljivači - imaju sve mogućnosti kao obični korisnici uz dodatne opcije da oglašavaju svoje dvorane, upravljaaju informacijama o dvorani, potvrđuju ili odbijaju termine

D4 -> administrator - osoba koja održava sustav te upravlja zahtjevima korisnika

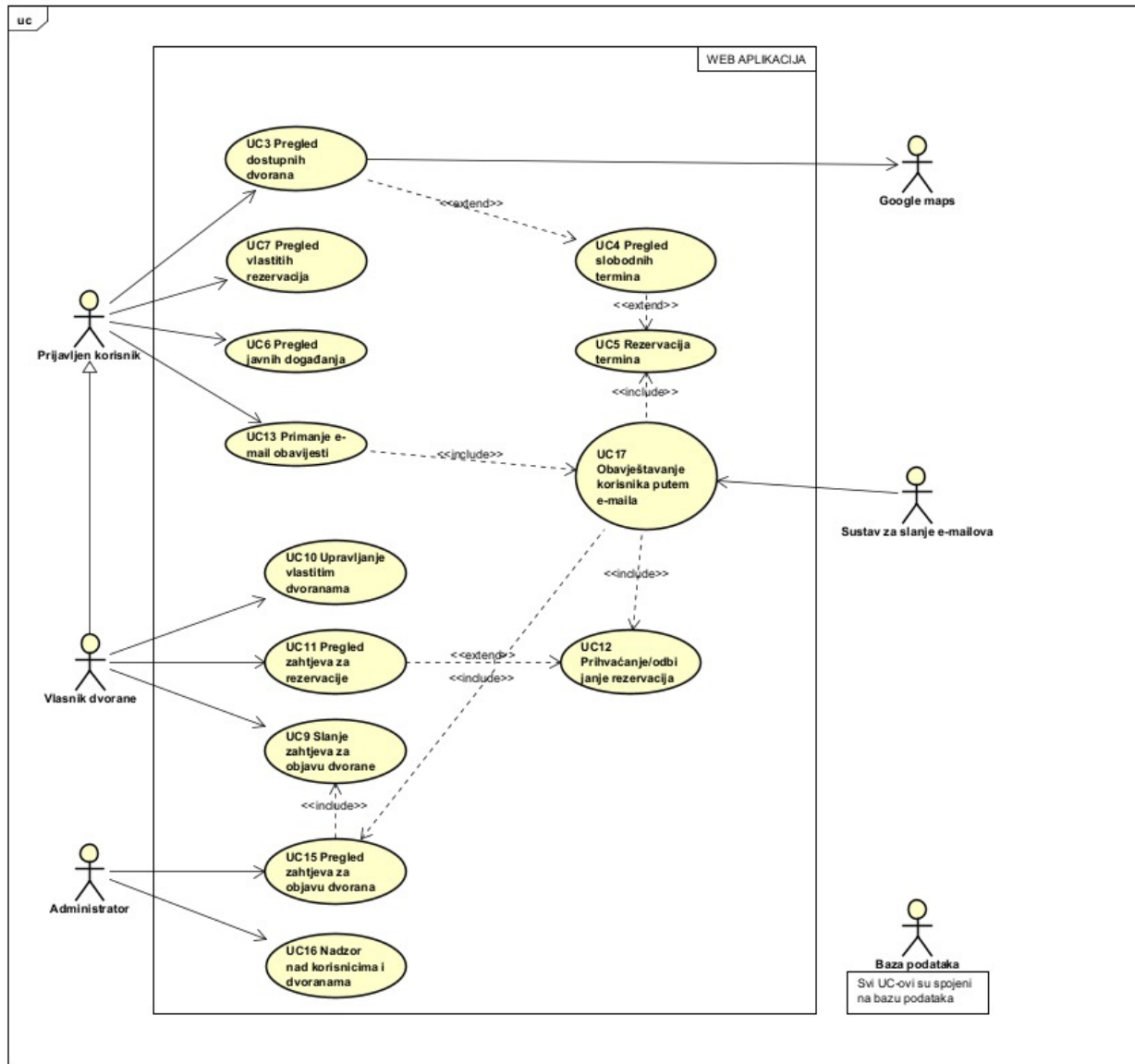
11.1 Aktori i njihovi funkcionalni zahtjevi

- A1 - Neprijavljeni korisnik (inicijator) > - Pregled početne stranice aplikacije (Home page) (FZ-03) > - Pregled popisa dvorana, oglasne ploče te karte s označenim dvoranama (FZ-03, FZ-04, FZ-05) > - Mogućnost registracije putem e-maila (FZ-01)
- A2 - Prijavljeni korisnik (inicijator) > - Registracija i prijava putem e-maila (FZ-01) > - Pregled dostupnih dvorana i slobodnih termina (FZ-06, FZ-19) > - Rezervacija termina (FZ-07) > - Definiranje namjene rezervacije (javno/privatno, broj gostiju) (FZ-08) > - Pregled i prijava na javna događanja > - Pregled i upravljanje vlastitim rezervacijama (FZ-09) > - Zahtjev za promjenom uloge u vlasnika dvorane (FZ-10)
- A3 - Vlasnik dvorane / Iznajmljivač (inicijator/sudionik) > - Sve mogućnosti običnog korisnika (FZ-01, FZ-06, FZ-07, FZ-08, FZ-09, FZ-10, FZ-19) > - Slanje zahtjeva za objavu dvorane (unesi opis, veličinu, kapacitet, cijenu, slike) (FZ-11, FZ-12) > - Upravljanje vlastitim oglašenim dvoranama (FZ-18) > - Pregled zahtjeva za rezervacije svojih dvorana (FZ-17) > - Prihvatanje ili odbijanje rezervacija (FZ-16) > - Primanje e-mail obavijesti o novim zahtjevima (FZ-17)
- A4 - Administrator (sudionik) > - Pregled i odobravanje zahtjeva za: > - Promjenu uloge korisnika u vlasnika dvorane (FZ-15) > - Objavu nove dvorane (FZ-14) > - Nadzor nad sustavom (uvid u korisnike, rezervacije, dvorane) (FZ-20) > - Održavanje sigurnosti i pouzdanosti aplikacije (FZ-20)

12 Obrasci uporabe

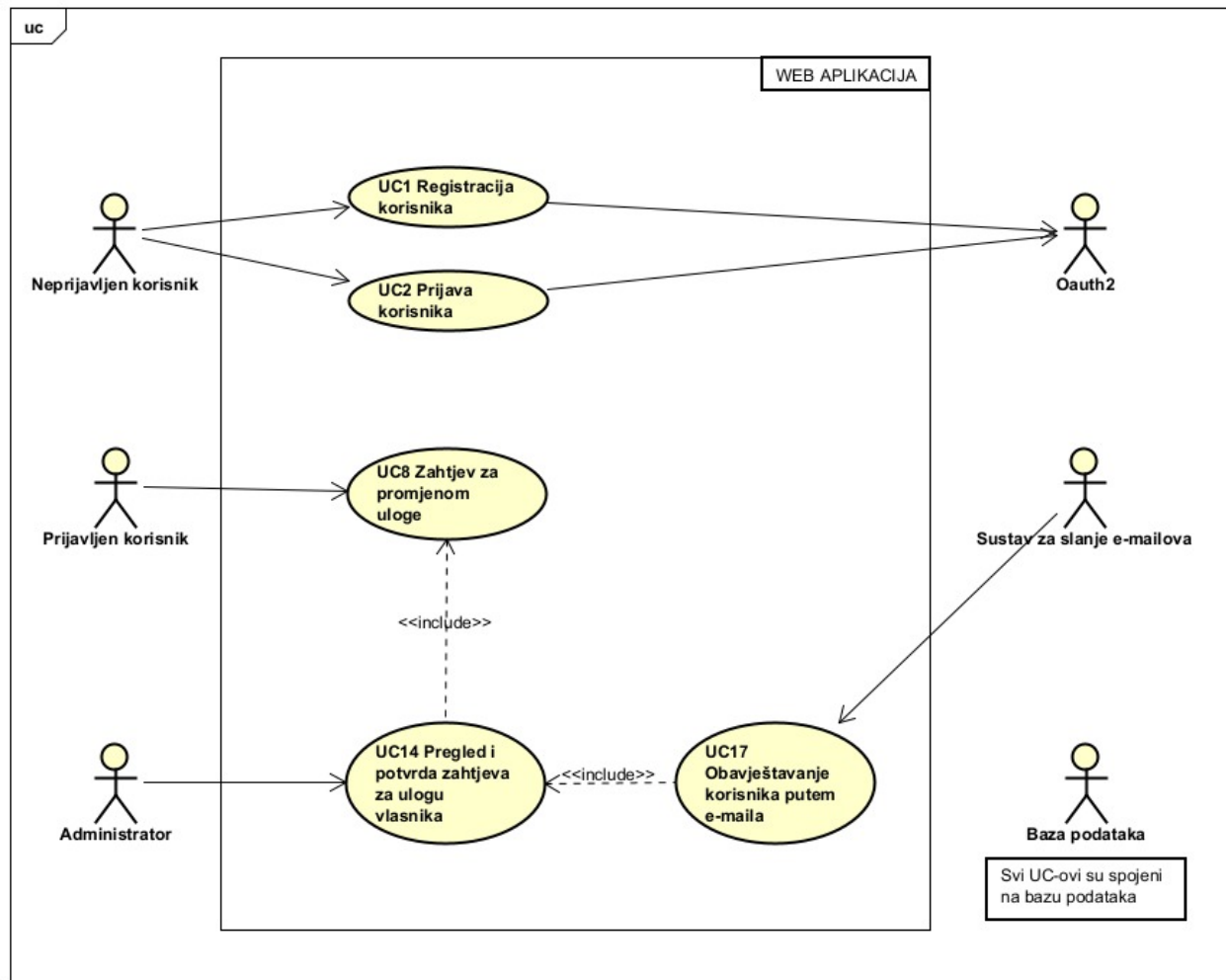
13 Dijagrami obrazaca uporabe

13.1 1. Visokorazinski dijagram obrazaca uporabe cijelog sustava



3.1 slika - Prikaz glavnih funkcionalnosti sustava Dijagram prikazuje dodatne pasivne sustave, glavne aktere i njihove glavne funkcionalnosti tijekom korištenja aplikacije te je obuhvaćena većina slučajeva korištenja.

13.2 2. dijagram obrazaca uporabe za ključne značajke



3.2 slika - Prikaz prijave i proces promjene uloge korisnika u iznajmljivača dvorane Dijagram prikazuje prijavu korisnika i proces promocije korisnika u iznajmljivača dvorane. Neprijavljen korisnik se registrira ukoliko se prvi put prijavljuje u aplikaciju te se mora pohraniti novi korisnik.

ID	Use Case naziv	Kratak opis
UC3	Pregled dostupnih dvorana	Prikaz svih objavljenih dvorana s terminima
UC4	Pregled slobodnih termina	Pregled slobodnih termina za odabranu dvoranu
UC5	Rezervacija termina	Rezervacija termina, unos svrhe, tipa događaja i broja gostiju
UC6	Pregled javnih događanja	Pregled javnih događanja objavljenih na oglasnoj ploči
UC7	Pregled vlastitih rezervacija	Pregled, izmjena ili otkazivanje vlastitih rezervacija
UC8	Zahtjev za promjenom uloge	Slanje zahtjeva za ulogu vlasnika dvorane
UC13	Primanje e-mail obavijesti	Sustav šalje e-mail obavijesti korisniku

13.3.3 3.3 Vlasnik dvorane (iznajmljivač)

ID	Use Case naziv	Kratak opis
UC9	Slanje zahtjeva za objavu dvorane	Slanje podataka o dvorani na odobravanje administratoru
UC10	Upravljanje vlastitim dvoranama	Uređivanje informacija o objavljenim dvoranama
UC11	Pregled zahtjeva za rezervacije	Pregled korisničkih zahtjeva za rezervaciju
UC12	Prihvaćanje/odbijanje rezervacija	Odluka o prihvaćanju ili odbijanju zahtjeva za rezervaciju

13.3.4 3.4 Administrator

ID	Use Case naziv	Kratak opis
UC14	Pregled zahtjeva za ulogu vlasnika	Pregled i odlučivanje o zahtjevima korisnika za ulogom vlasnika dvorane
UC15	Pregled zahtjeva za objavu dvorana	Odobravanje ili odbijanje prijave novih dvorana
UC16	Nadzor nad korisnicima i dvoranama	Pregled korisnika, dvorana i sustavnih aktivnosti

13.3.5 3.5 Sustav e-pošte (automatizirani akter)

ID	Use Case naziv	Kratak opis
UC17	Obavješćavanje korisnika putem e-maila	Automatsko slanje e-mail obavijesti korisnicima

14 Opis obrazaca uporabe

14.1 UC1 – Registracija korisnika

Glavni sudionik: Neprijavljeni korisnik

Cilj: Prva prijava u aplikaciju pomoću Google accounta

Sudionici: Baza podataka, OAuth2

Preduvjet: Korisnik se nikada prije nije prijavio s tim Google računom

- Osnovni tijek:** 1. Korisnik otvara stranicu za prijavu
2. Klikne gumb za Google prijavu
3. Nakon autentifikacije, vraća se u aplikaciju prijavljen
4. Sustav sprema korisnika u bazu podataka
-

14.2 UC2 – Prijava korisnika

Glavni sudionik: Neprijavljeni korisnik

Cilj: Prijava u aplikaciju pomoću Google accounta

Sudionici: Baza podataka, OAuth2

Preduvjet: Korisnik je već ranije registriran

- Osnovni tijek:** 1. Korisnik otvara stranicu za prijavu
2. Klikne Google prijavu
3. Nakon autentifikacije vraća se u aplikaciju
4. Sustav učitava podatke povezane s korisnikom
-

14.3 UC3 – Pregled dostupnih dvorana

Glavni sudionik: Korisnik

Cilj: Pregled svih objavljenih dvorana

Sudionici: Baza podataka, Google Maps API

Preduvjet: Korisnik pregledava početnu stranicu web-aplikacije

- Osnovni tijek:** 1. Korisnik otvara popis dvorana
2. Sustav dohvaća i prikazuje dvorane (opis, slike, cijena)
3. Korisnik može filtrirati rezultate (lokacija, veličina, cijena)
4. Korisnik pritisne na karticu “Karta” i prikažu mu se sve dvorane na karti pomoću servisa Google Maps API

Odstupanja: - 2a. Nema dostupnih dvorana → poruka „Trenutno nema dostupnih dvorana.”

- 3a. Ukoliko korisnik nije prijavljen, sustav ga preusmjerava na stranicu za login —

14.4 UC4 – Pregled slobodnih termina

Glavni sudionik: Prijavljen Korisnik

Cilj: Pregledati slobodne termine određene dvorane

Sudionici: Baza podataka

Preduvjet: Dvorana je objavljena i ima upisan raspored termina te korisnik mora biti prijavljen

- Osnovni tijek:** 1. Korisnik odabire dvoranu
2. Sustav dohvaća slobodne termine i prikazuje ih pomoću kalendara 3. Prikazuju se datumi, sati i cijene
-

14.5 UC5 – Rezervacija termina

Glavni sudionik: Prijavljen Korisnik

Cilj: Rezervirati željeni termin

Sudionici: Vlasnik dvorane, Sustav e-pošte, Baza podataka

Preduvjet: Korisnik je prijavljen i termin je slobodan

- Osnovni tijek:** 1. Korisnik odabire slobodan termin
2. Ispunjava formu (svrha, broj gostiju, tip događaja)
3. Potvrđuje rezervaciju

4. Sustav sprema rezervaciju u bazu
5. Sustav šalje e-mail vlasniku

Odstupanja: - 4a. Termin zauzet → „Termin više nije dostupan.”

14.6 UC6 – Pregled javnih događanja

Glavni sudionik: Prijavljen Korisnik

Cilj: Vidjeti javna događanja drugih korisnika

Sudionici: Baza podataka

Preduvjet: Postoje objavljena javna događanja

Osnovni tijek: 1. Korisnik otvara oglasnu ploču
2. Sustav prikazuje sva javna događanja

14.7 UC7 – Pregled vlastitih rezervacija

Glavni sudionik: Prijavljen Korisnik

Cilj: Vidjeti, izmijeniti ili otkazati vlastite rezervacije

Sudionici: Baza podataka

Preduvjet: Korisnik je prijavljen u sustav

Osnovni tijek: 1. Korisnik otvara svoj profil
2. Sustav prikazuje sve njegove rezervacije

Odstupanja: - 4a. Nema rezerviranih termina → „Nije rezerviran niti jedan termin”

14.8 UC8 – Zahtjev za promjenom uloge

Glavni sudionik: Prijavljen Korisnik

Cilj: Zatražiti status vlasnika dvorane

Sudionici: Administrator, Baza podataka

Preduvjet: Korisnik je prijavljen kao običan korisnik

Osnovni tijek: 1. Korisnik ispunjava zahtjev
2. Sustav sprema i prosljeđuje zahtjev administratoru
3. Administrator odlučuje (UC14)

14.9 UC9 – Slanje zahtjeva za objavu dvorane

Glavni sudionik: Vlasnik dvorane

Cilj: Dodati novu dvoranu u sustav

Sudionici: Administrator, Baza podataka

Preduvjet: Korisnik ima ulogu vlasnika dvorane

Osnovni tijek: 1. Vlasnik unosi podatke (opis, kapacitet, slike, cijena)
2. Sustav sprema i šalje zahtjev administratoru
3. Administrator odobrava/odbija (UC15)

14.10 UC10 – Upravljanje vlastitim dvoranama

Glavni sudionik: Vlasnik dvorane

Cilj: Uređivanje svojih objavljenih dvorana

Sudionici: Baza podataka

Preduvjet: Dvorana je već objavljena

Osnovni tijek: 1. Vlasnik otvara listu svojih dvorana
2. Odabire dvoranu
3. Mijenja informacije
4. Sustav sprema izmjene

14.11 UC11 – Pregled zahtjeva za rezervacije

Glavni sudionik: Vlasnik dvorane

Cilj: Pregledati sve zaprimljene zahtjeve za svoje dvorane

Sudionici: Baza podataka

Preduvjet: Postoje zahtjevi

Osnovni tijek: 1. Sustav prikazuje sve zahtjeve
2. Vlasnik bira zahtjev
3. Nastavlja s UC12 (odbij ili prihvati)

14.12 UC12 – Prihvaćanje / odbijanje rezervacija

Glavni sudionik: Vlasnik dvorane

Cilj: Donijeti odluku o rezervaciji

Sudionici: Sustav e-pošte, Baza podataka

Preduvjet: Postoji zahtjev

Osnovni tijek: 1. Vlasnik odabire zahtjev
2. Odluka se bilježi u bazu
3. Sustav šalje e-mail korisniku

14.13 UC13 – Primanje e-mail obavijesti

Glavni sudionik: Korisnik

Cilj: Primanje obavijesti o promjenama/rezervacijama

Preduvjet: Obavijest je poslana

Osnovni tijek:

- Korisnik prima e-mail obavijest

14.14 UC14 – Pregled zahtjeva za ulogu vlasnika

Glavni sudionik: Administrator

Cilj: Odobriti ili odbiti zahtjeve za promjenu uloge

Sudionici: Baza podataka

Preduvjet: Postoje zahtjevi

- Osnovni tijek:** 1. Administrator pregleda zahtjev
2. Odobri ili odbije
3. Sustav šalje e-mail korisniku
-

14.15 UC15 – Pregled zahtjeva za objavu dvorana

Glavni sudionik: Administrator

Cilj: Odluka o objavi novih dvorana

Sudionici: Baza podataka

Preduvjet: Zaprimljeni zahtjevi vlasnika

- Osnovni tijek:** 1. Administrator pregleda zahtjeve
2. Donosi odluku
3. Sustav šalje obavijest vlasniku
-

14.16 UC16 – Nadzor nad korisnicima i dvoranama

Glavni sudionik: Administrator

Cilj: Nadgledati sustav, dvorane, korisnike

Sudionici: Baza podataka

Preduvjet: Administrator je prijavljen

- Osnovni tijek:** 1. Otvara administracijsku ploču
2. Pregledava korisnike, dvorane, rezervacije
3. Može reagirati (brisanje, blokiranje itd.)
-

14.17 UC17 – Obavješćavanje korisnika putem e-maila

Glavni sudionik: Sustav e-pošte

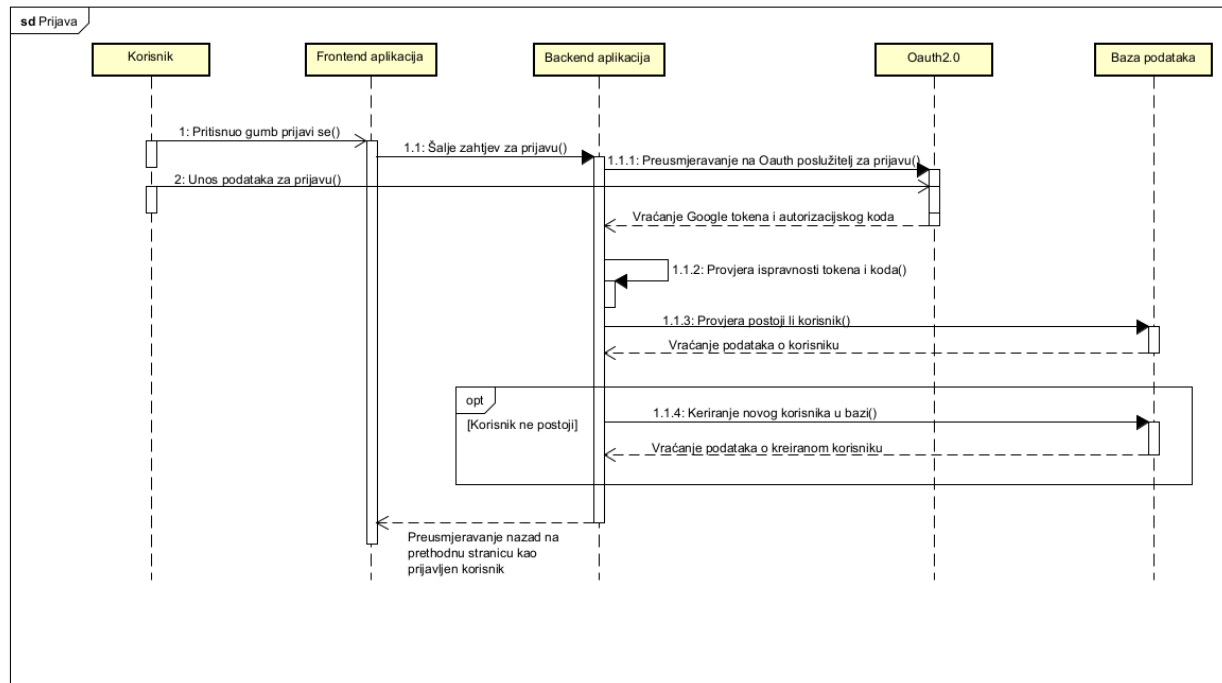
Cilj: Automatski slati obavijesti korisnicima

Sudionici: Baza podataka

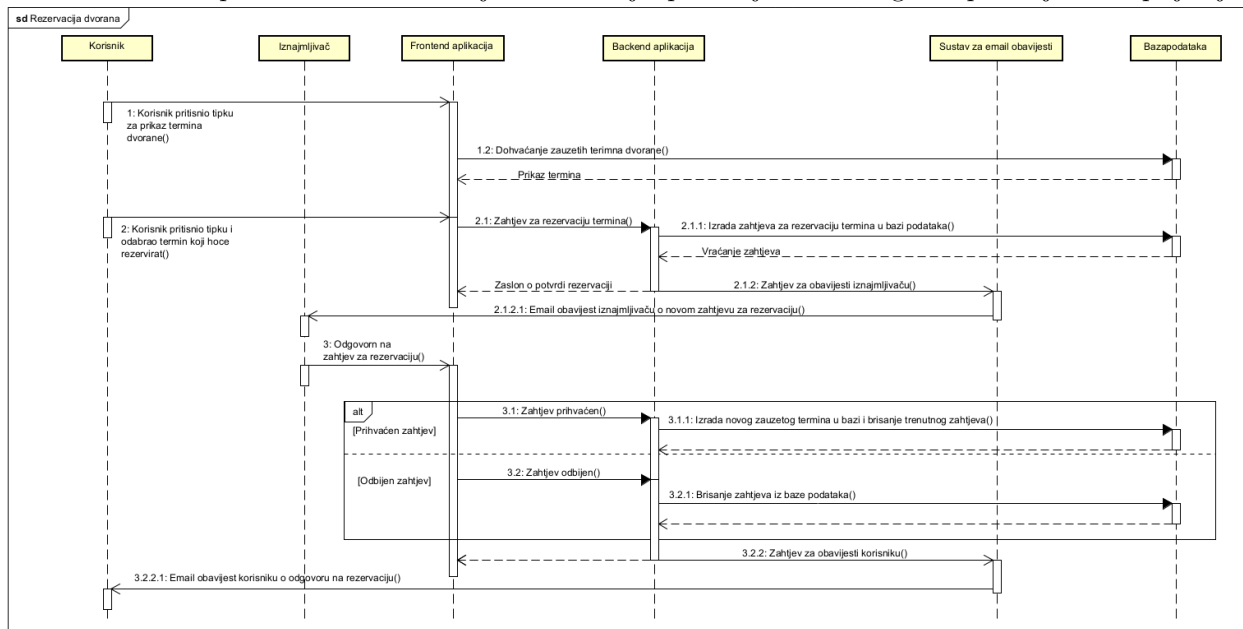
Preduvjet: Nastao je događaj koji zahtijeva e-mail obavijest

- Osnovni tijek:** 1. Sustav detektira potrebnu obavijest
2. Generira e-mail
3. Automatski šalje korisniku
-

15 Sekvencijski dijagrami

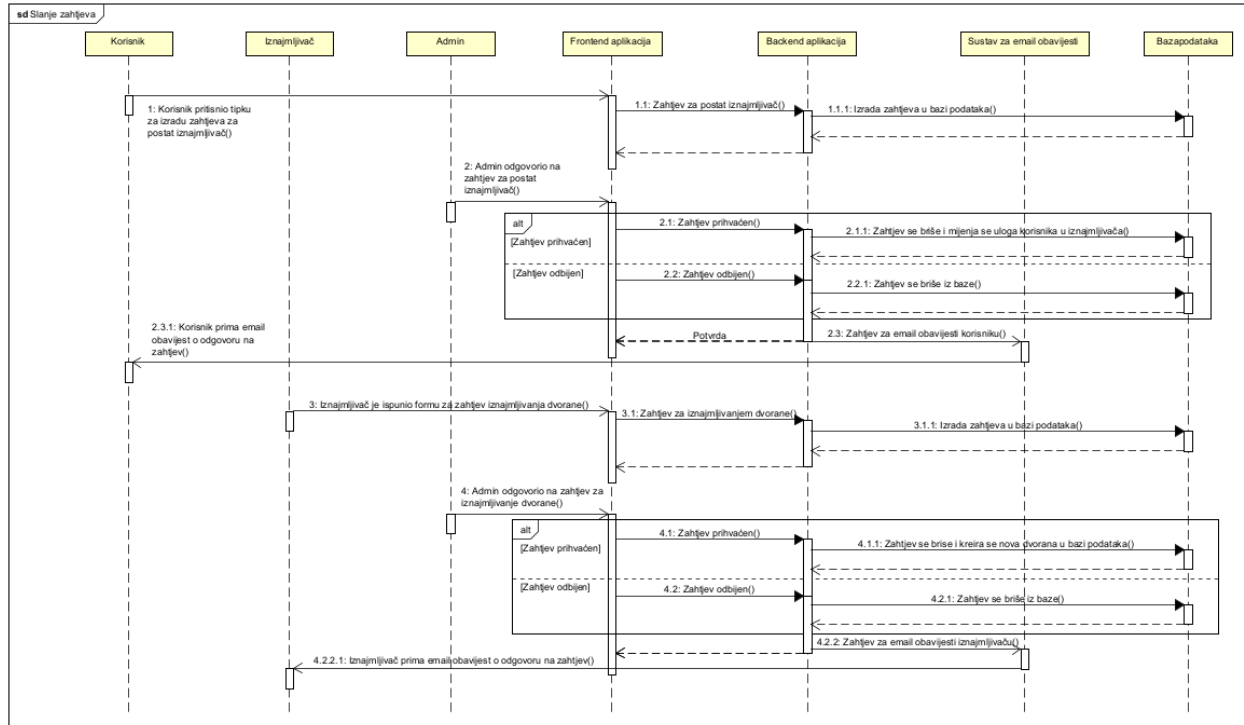


Slika 3.4 - Sekvencijski dijagra za prijavu korisnika Sekvencijski dijagram prikazuje postupak prijave korisnika putem Google OAuth2 autentifikacije. Korisnik pritisne gumb za prijavu, a frontend aplikacija šalje zahtjev backendu koji preusmjerava korisnika na OAuth poslužitelj. Nakon uspješne prijave, poslužitelj vraća token i autorizacijski kod backendu koji provjerava njihovu ispravnost te traži korisnika u bazi. Ako korisnik postoji, vraćaju se njegovi podaci, a ako ne, kreira se novi zapis u bazi. Na kraju korisnik je preusmjeren natrag u aplikaciju kao prijavljen.



Slika 3.5 - Sekvencijski dijagra za prikaz funkcionalnosti rezervacije dvorane Sekvencijski dijagram „Rezervacija dvorana“ prikazuje proces rezervacije termina između korisnika, iznajmljivača i sustava. Korisnik putem aplikacije prvo pregledava slobodne termine koje backend dohvaća iz baze podataka i Google kalendara.

Nakon što odabere željeni termin, frontend šalje zahtjev backendu, koji pohranjuje zahtjev o rezervaciji i obavještava iznajmljivača putem emaila. Iznajmljivač zatim odlučuje hoće li zahtjev prihvatiti ili odbiti. U slučaju prihvatanja, backend kreira zauzeti termin u bazu, ako je odbijen, zahtjev se briše. Na kraju, sustav putem email obavijesti javlja korisniku odgovor na zahtjev rezervacije. Proces jasno prikazuje međusobnu komunikaciju i automatizirano upravljanje terminima.



Slika 3.6 - Sekvencijski dijagram za prikaz funkcionalnosti slanja zahtjeva administratoru i odgovaranja na zahtjeve. Sekvencijski dijagram „Slanje zahtjeva“ prikazuje proces u kojem korisnik i iznajmljivač šalju zahtjeve administratoru, a sustav obrađuje i obavještava sudionike o ishodu. Korisnik prvo šalje zahtjev da postane iznajmljivač; frontend ga prosljeđuje backendu koji ga sprema u bazu podataka. Administrator zatim odlučuje hoće li zahtjev prihvatiti ili odbiti. Ako je prihvaćen, korisniku se mijenja uloga u iznajmljivača, a ako nije, zahtjev se briše. Nakon toga sustav šalje korisniku email obavijest o odluci. Nakon što postane iznajmljivač, korisnik može poslati zahtjev za iznajmljivanje dvorane. Proces je sličan — zahtjev se sprema u bazu, administrator ga odobrava ili odbija, ukoliko je odobren se kreira nova dvorana za iznajmljivanje, a ako nije samo se zahtjev briše. Sustav na kraju automatski šalje iznajmljivaču email obavijest s odgovorom. Dijagram jasno prikazuje automatiziran i transparentan tijek obrade zahtjeva i obavještavanja.

16 Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

ID obrasca uporabe	Naziv obrasca uporabe	Povezani funkcionalni zahtjevi	Obrazloženje poveznice
UC1	Registracija korisnika	F-01, F-02	Registracija putem Google računa (OAuth2) i inicijalno dodjeljivanje uloge običnog korisnika.
UC2	Prijava korisnika	F-01, F-02	Prijava putem Google računa i prepoznavanje uloge korisnika.

ID obrasca uporabe	Naziv obrasca uporabe	Povezani funkcionalni zahtjevi	Obrazloženje poveznice
UC3	Pregled dostupnih dvorana	F-03, F-05, F-06, F-19	Prikaz popisa dvorana, prikaz na karti, ograničen pristup neprijavljenih korisnika i mogućnost filtriranja.
UC4	Pregled slobodnih termina	F-06, F-13	Pregled detalja dvorane i prikaz dostupnih termina s cijenama.
UC5	Rezervacija termina	F-07, F-08, F-13, F-17	Rezervacija termina, unos potrebnih podataka, provjera dostupnosti i automatsko obavješćavanje vlasnika putem e-maila.
UC6	Pregled javnih događanja	F-04	Prikaz oglasne ploče sa svim javnim događanjima.
UC7	Pregled vlastitih rezervacija	F-09	Pregled, izmjena i otkazivanje vlastitih rezervacija korisnika.
UC8	Zahtjev za promjenom uloge	F-10, F-15	Korisnik može zatražiti promjenu uloge u vlasnika, a administrator odlučuje.
UC9	Slanje zahtjeva za objavu dvorane	F-11, F-12, F-14	Vlasnik unosi podatke o dvorani, sustav ih validira i prosljeđuje administratoru na odobrenje.
UC10	Upravljanje vlastitim dvoranama	F-18, F-12	Vlasnik uređuje informacije o svojim objavljenim dvoranama (opis, slike, cijene, termini).
UC11	Pregled zahtjeva za rezervacije	F-17, F-18	Pregled zaprimljenih zahtjeva za rezervacije povezan je s e-mail obavijestima i uređivanjem statusa zahtjeva.
UC12	Prihvatanje / odbijanje rezervacija	F-17	Sustav šalje e-mail obavijesti korisniku o odluci vlasnika te obavještava obje strane.
UC13	Primanje e-mail obavijesti	F-16	Automatsko slanje e-mail obavijesti korisnicima o rezervacijama i promjenama.
UC14	Pregled zahtjeva za ulogu vlasnika	F-15	Administrator odobrava ili odbija zahtjeve za promjenu uloge korisnika.

ID obrasca uporabe	Naziv obrasca uporabe	Povezani funkcionalni zahtjevi	Obrazloženje poveznice
UC15	Pregled zahtjeva za objavu dvorana	F-14	Administrator odlučuje o objavi dvorana i obavještava vlasnika o odluci.
UC16	Nadzor nad korisnicima i dvoranama	F-20	Administrator nadzire korisnike, dvorane i rezervacije te može uređivati ili brisati zapise.
UC17	Obavještavanje korisnika putem e-maila	F-16, F-17	Sustav automatski generira i šalje e-mail obavijesti o događajima u sustavu.

17 Arhitektura sustava

17.1 Opis arhitekture

17.1.1 Stil arhitekture

Izabrali smo stil klijent-poslužitelj. Razlozi biranja ovog stila navedeni su u nastavku.

17.1.2 Podsustavi

Podsustavi koji se koriste u aplikaciji su OAuth2, Maps Embed API i Cloudinary. OAuth2 koristi se za autentifikaciju korisnika i provjeravanje lozinki prilikom login-a. Maps Embed API se koristi za prikazivanje lokacija dvorana na karti. Cloudinary se koristi kao servis za spremanje, ažuriranje i posluživanje slika koje koristimo u aplikaciji.

17.1.3 Preslikavanje na radnu platformu

Arhitektura će biti implementirana na platformi Render jer je besplatna i omogućuje nam otvaranje servisa za aplikaciju i servisa za bazu podataka.

17.1.4 Spremišta podataka

Za pohranu podataka koristimo relacijsku bazu podataka koja je detaljnije objašnjena u nastavku.

17.1.5 Mrežni protokoli

Za komunikaciju između komponenata sustava koristiti će se protokol HTTP.

17.1.6 Globalni upravljački tok

Korisničko sučelje za svaku korisničku interakciju sa sustavom šalje upit programskoj potpori sustava. Ona obrađuje taj upit i po potrebi pristupa bazi podataka kako bi dohvatila neke podatke. Nakon obrade, traženi resursi se vraćaju korisničkom sučelju, koje ih tada može prikazati korisniku.

17.1.7 Sklopovskoprogamski zahtjevi

Aplikaciju će biti moguće koristiti bez obzira na operacijski sustav i na više različitih uređaja (stolno računalo, prijenosno računalo, prijenosni telefon...). Sve što je potrebno jest da uređaj na kojem se pokreće aplikacija ima web preglednik koji podržava HTML, CSS i Javascript. Uz to, potrebno je da uređaj ima stabilnu internetsku vezu s *download* i *upload* brzinama od barem 1Mb po sekundi.

17.2 Obrazloženje odabira arhitekture

Arhitektura klijent-poslužitelj je najčešće korištena arhitektura kod izrade web aplikacija. Izabrali smo ju jer nudi jasnu podjelu na klijenta i poslužitelja što se slagalo s potrebama našeg sustava.

17.2.1 Izbor arhitekture temeljen na principima oblikovanja

Arhitektura klijent-poslužitelj bila je dobar izbor za naš sustav zbog njene niske povezanosti. Klijent(frontend) i poslužitelj(backend) nisu povezani, neovisni su jedan o drugome. To nam omogućuje razvoj oba dijela sustava u isto vrijeme i olakšava pronalazak izvora greški. Ovaj stil također je fleksibilan i zato nam je lakše dodati novu funkcionalnost u sustav. Još jedna prednost ovog stila je visoka kohezija koja nam je omogućila da podijelimo zadatke na puno podzadataka radi lakšeg razumijevanja, testiranja i ispravljanja grešaka.

17.2.2 Razmatrane alternative

Odmah smo se složili da nam je arhitektura klijent-poslužitelj najbolji izbor za aplikaciju pa nismo razmatrali druge alternative.

17.3 Organizacija sustava na visokoj razini

17.3.1 Klijent-poslužitelj

Sustav je organiziran tako da je korisničko sučelje(frontend) klijent, a programska potpora(backend) poslužitelj.

17.3.2 Baza podataka

Bazu podataka koristimo za spremanje podataka o svim dvoranama i njihovim lokacijama. Lokacije su spremljene pomoću njihovih koordinata i adresa i povezane s gradom u kojem se nalaze. Uz dvorane vežu se kategorije koje nam govore za što je dvorana namijenjena i njihovi iznajmljivači. Nadalje, u bazu spremamo zauzete termine koje vezemo uz određene dvorane i korisnike koji su ih rezervirali. Svi iznajmljivači i korisnici spremljeni su skupa s adminima u tablici. Također, spremamo podatke o zahtjevima za mijenjanje uloge iz korisnika u iznajmljivača i zahtjeve za oglašavanje novih dvorana.

17.3.3 Grafičko sučelje

Klijent informacije o dvoranama i korisnicima dobiva zahtjevima koje šalje na poslužitelj. Kada klijent primi odgovor on formatira informacije odgovora te ih adekvatno prikaže na grafičkom sučelju

Grafičko sučelje je jednostavno organizirano kako bi ga korisnici svih razina tehničke pismenosti mogli koristiti. Na početnoj stranici je vidljiva lista dostupnih dvorana te gumb za prijavu korisnika, gumb za prelazak na stranicu oglasne ploče te gumb za prelazak na kartu s označenim dvoranama.

Kada je korisnik logiran pritiskom na gumb u gornjem lijevom kutu (svoju sliku profila) sa početne stranice, oglasne ploče ili mape, pristupa svom profilu gdje vidi svoje podatke te podatke o svojim dvoranama i rezervacijama.

Prikaz profila iznajmljivača. Uz prikazane rezervacije, vidimo i prikaz iznajmljivačevih dvorana te mogućnost uređivanja tih dvorana.

Pritiskom na gumb “Objavi dvoranu” na profilnoj stranici, korisniku se prikazuje ovakav prikaz te može poslati zahtjev kako bi objavio svoju dvoranu.

Ovo je stranica administratora. Administrator ovdje ima pristup gumbu za dodavanje kategorija, gumbu za brisanje dvorana i korisnika te pregled novih zahtjeva za iznajmljivače ili zahtjeva za oglašavanjem novih dvorana

Pritiskom na gumb “Karta” korisnik vidi prikaz karte grada Zagreba te na njoj označene lokacije dostupnih dvorana.

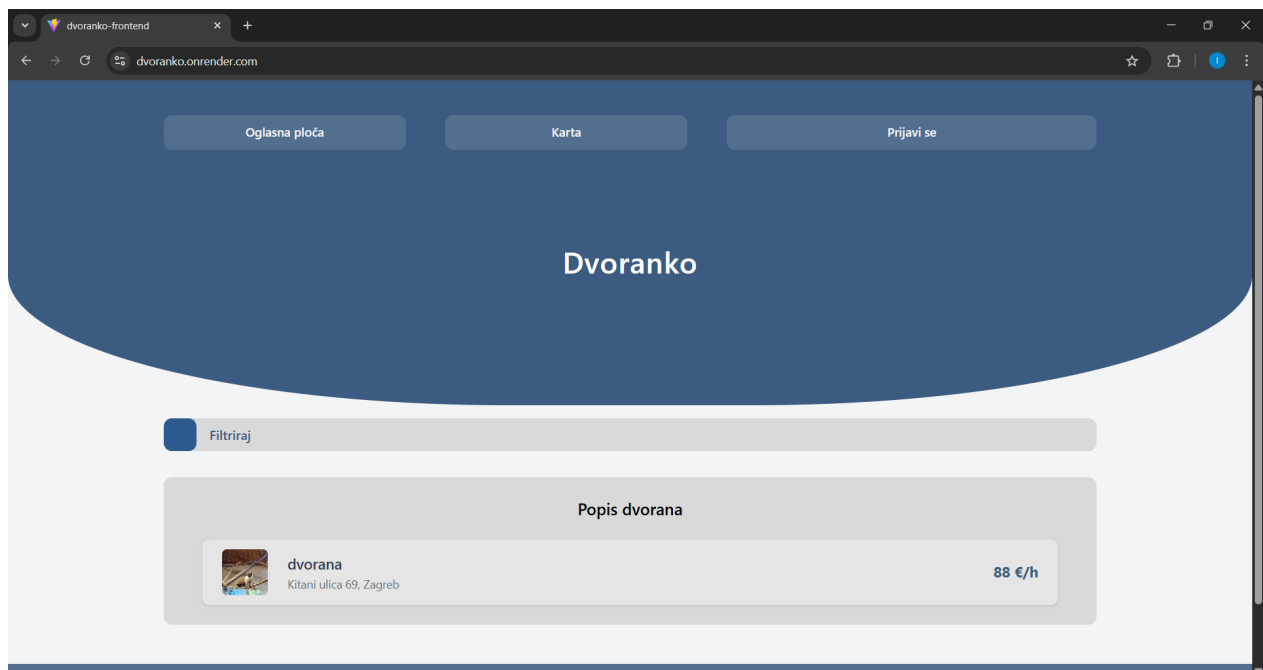


Figure 3: Slika

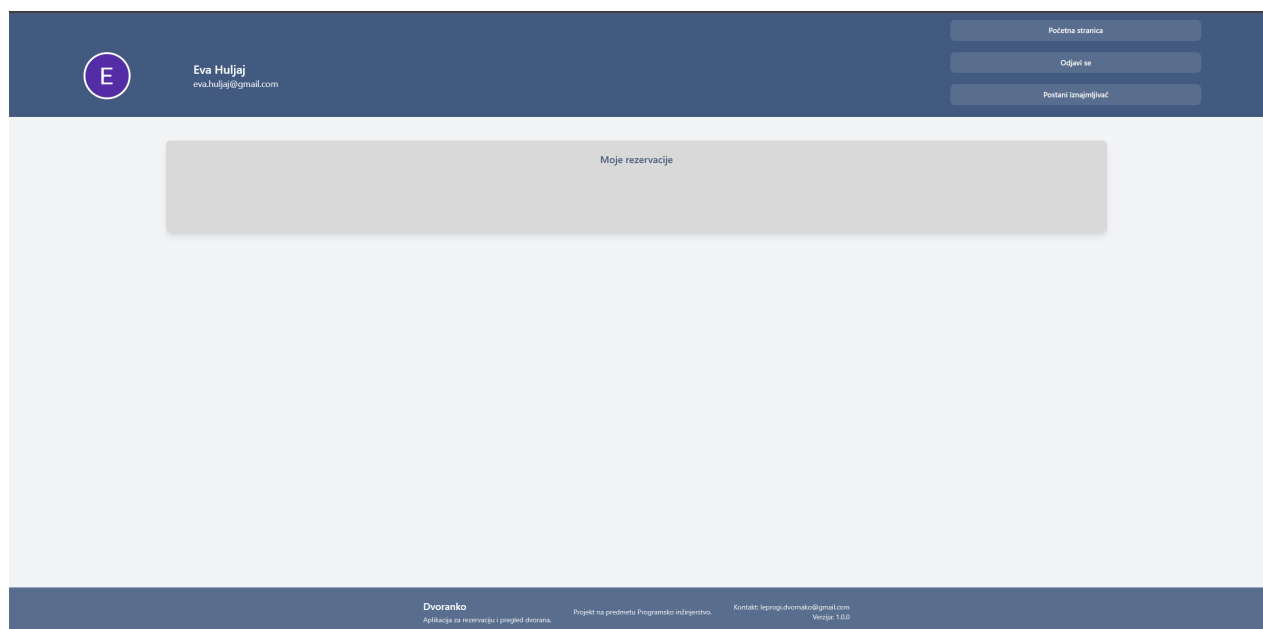


Figure 4: Slika

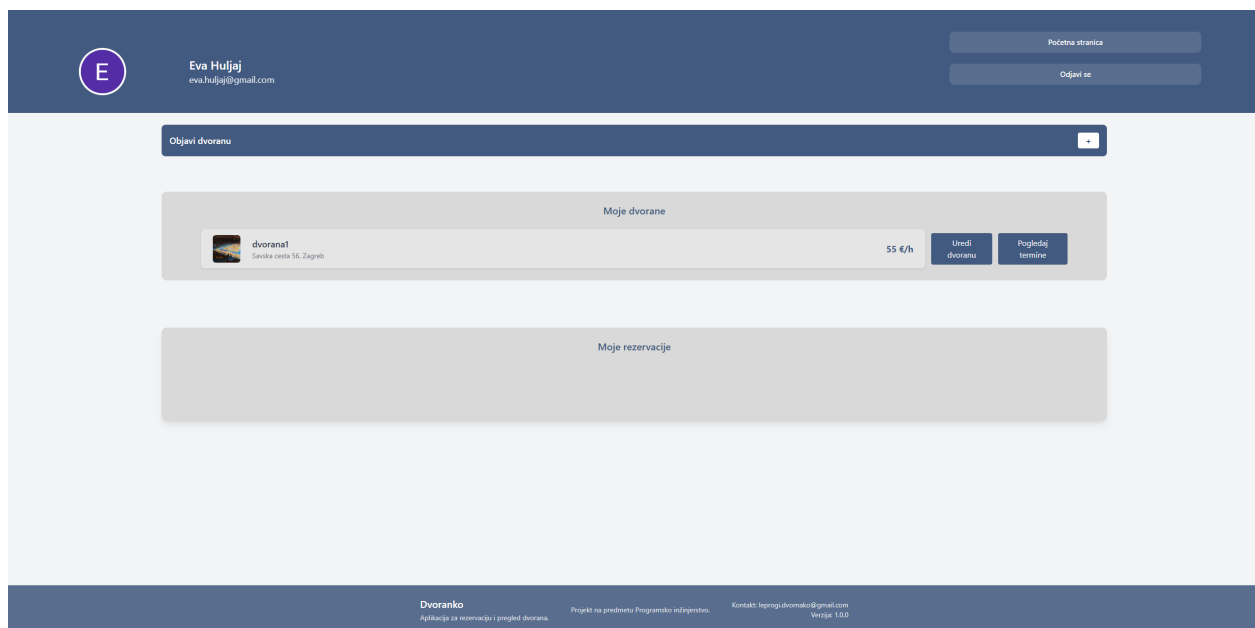


Figure 5: Slika

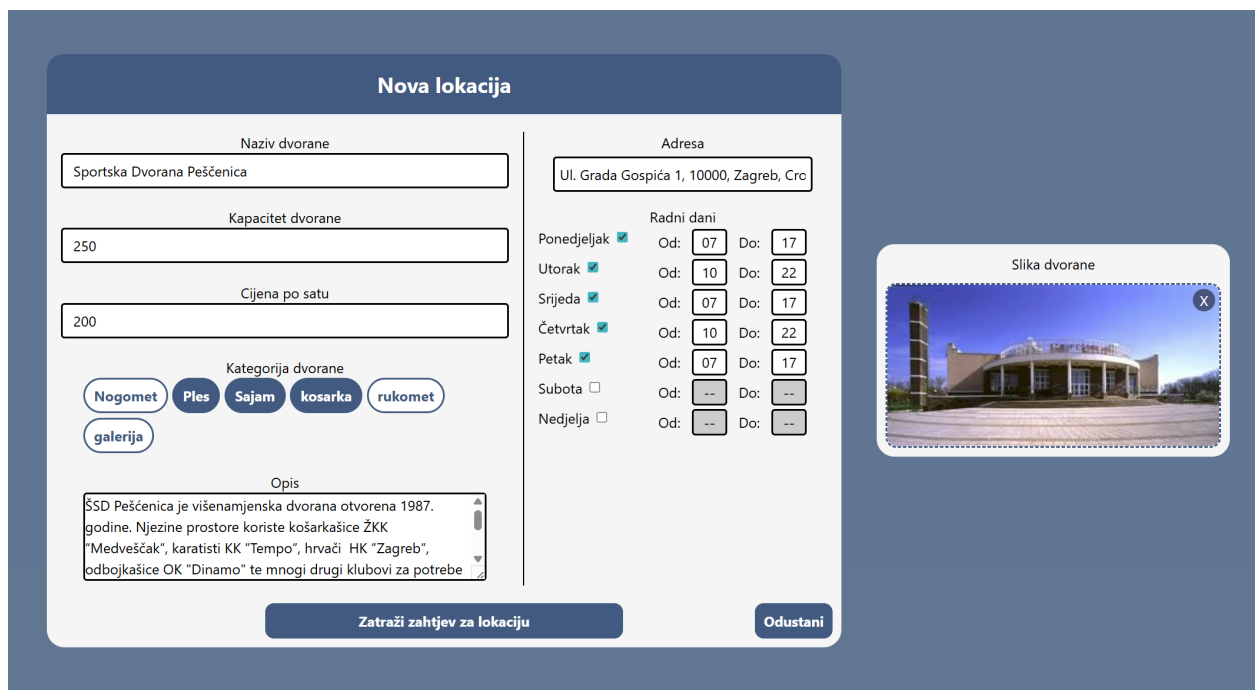


Figure 6: Slika

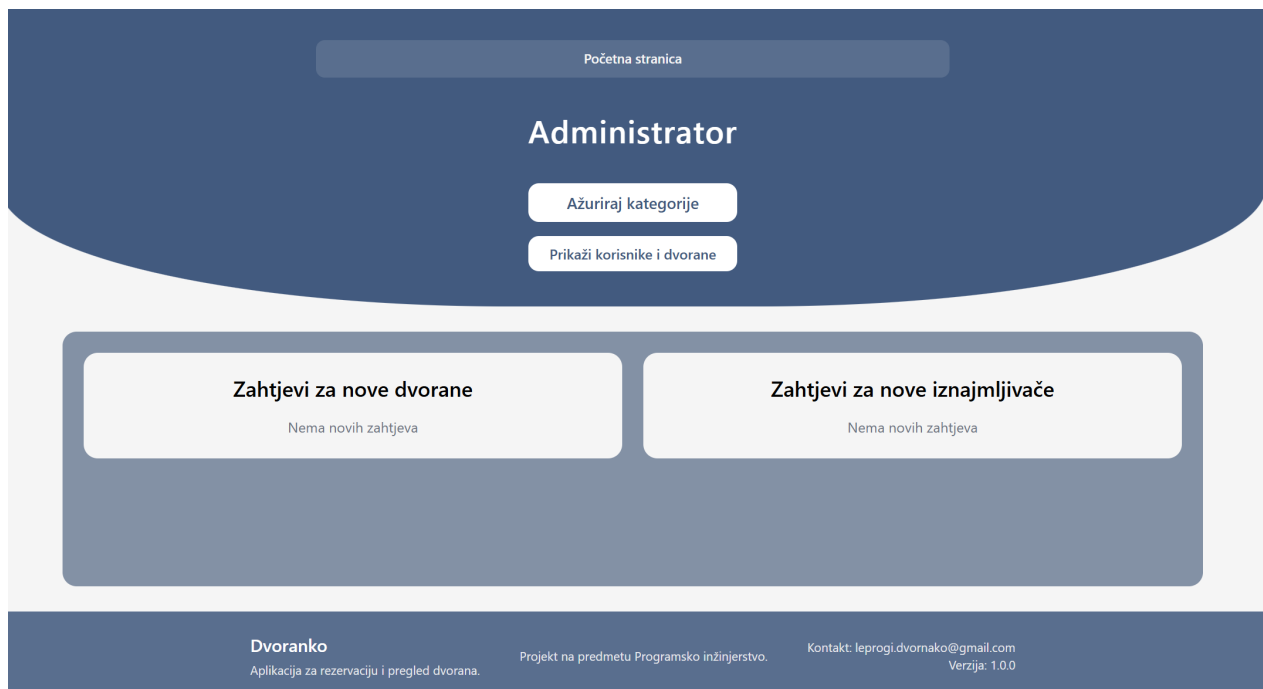


Figure 7: Slika

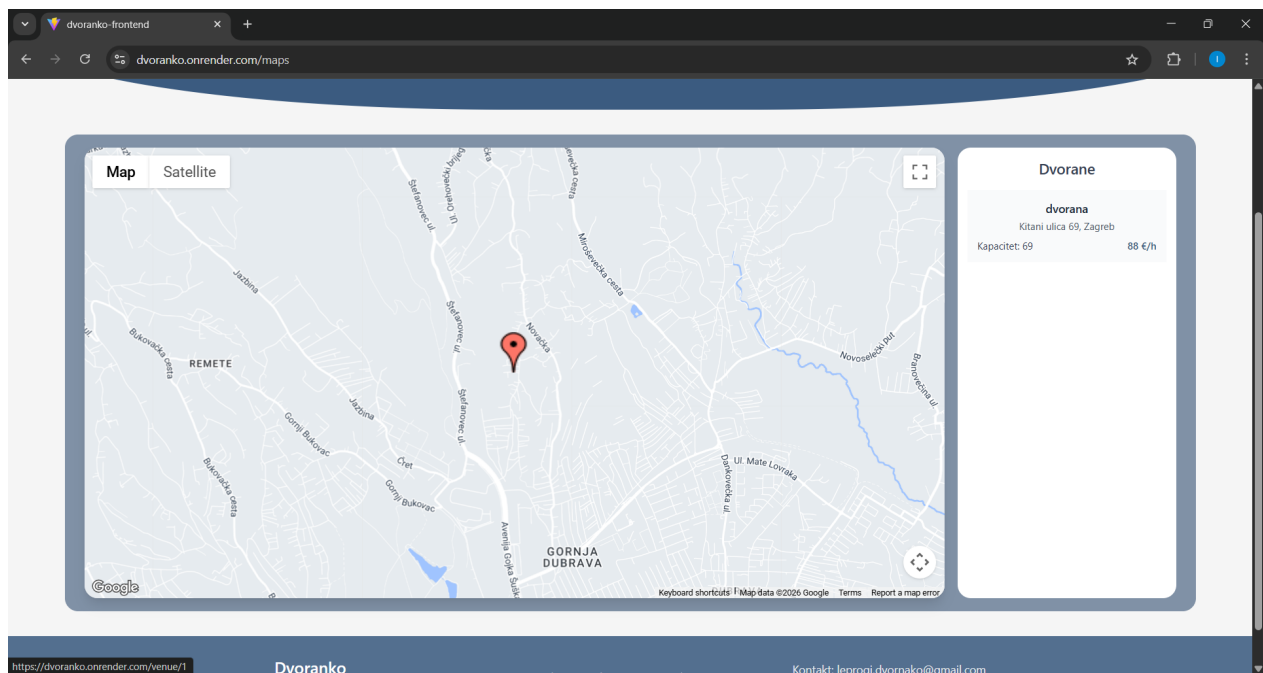


Figure 8: Slika

17.4 Organizacija aplikacije

17.4.1 Frontend i Backend slojevi

Aplikacija Dvoranko koristi frontend i backend sloj koji međusobno komuniciraju putem REST API-ja.

Frontend sloj sastoji se od prezentacijskog sloja čija je funkcija upravljanje korisničkim sučeljem i interakcijom korisnika s aplikacijom. Sastoji se od HTML-a, CSS-a i JS frameworka React.

Backend sloj sastoji se od poslovnog sloja i podatkovnog sloja. Poslovni sloj sadrži logiku aplikacije i ona obrađuje svu temeljnu logiku aplikacije, tj. određuje kako će se podaci obrađivati i slati nazad korisniku ili u bazu podataka. Za poslovni sloj korišten je Java framework Spring Boot. Podatkovni sloj zaslužan je za direktnom komunikacijom s bazom podataka. Njegov posao je spremanje i dohvaćanje podataka. Za podatkovni sloj korišten je Spring Data JPA koji komunicira s bazom podataka koja je izrađena u PostgreSQL-u.

18 Baza podataka

Podatci su spremljeni u relacijskoj bazi podataka PostgreSQL. Ona je implementirana pomoću Render servisa. Aplikacija se na bazu povezuje putem SQL upita. Glavne komponenta baze su entiteti Users, Dvorana, Slika_dvorana, Kategorija, Dvorana_kategorija, Termin, Adresa, Mjesto, Zahtjev_oglas, Zahtjev_oglas_kategorija, Zahtjev_slika, Zahtjev_iznajmljivac i Zahtjev_termin. Za navedene entitete stvorene su tablice opisane u nastavku. One su spojene preko primarnih i stranih ključeva.

18.1 Opis tablica

18.1.1 Users

18.1.1.1 Sadrži podatke o svim korisnicima, iznajmljivačima i adminima

Atribut	Tip podatka	Opis varijable
id	BIGINT (Primarni ključ)	Jedinstveni identifikator korisnika
name	VARCHAR(255)	Ime korisnika
picture_url	VARCHAR(255)	Poveznica na sliku korisnikovog goolge računa
role	VARCHAR(255)	Uloga korisnika(korisnik, iznajmljivač, admin)
email	VARCHAR(255)	E-mail adresa korisnika
google_id	VARCHAR(255)	Google id korisnikovog goolge računa

18.1.2 Dvorana

18.1.2.1 Sadrži podatke o svim dvoranama

Atribut	Tip podatka	Opis varijable
id_dvorana	BIGINT (Primarni ključ)	Jedinstveni identifikator dvorane
naziv_dvorana	VARCHAR(255)	Naziv dvorane
kapacitet	INTEGER	Kapacitet dvorane
opis	VARCHAR(255)	Opis dvorane (specifična namjena, kontakt iznajmljivača, cijena rezervacije, ...)
days_open	VARCHAR(255)	Podatci o tome u kojim satima za svaki dan u tjednu iznajmljivač želi iznajmljivati dvoranu
id_adresa	BIGINT (Strani ključ)	Identifikator adrese na kojoj se nalazi dvorana
cijena_po_satu	REAL	Cijena najma dvorane za jedan sat
id_vlasnik	BIGINT (Strani ključ)	Identifikator korisnika (iznajmljivača) koji oglašava dvoranu

18.1.3 Dvorana_kategorija

18.1.3.1 Sadrži veze između dvorana i njihovih kategorija

Atribut	Tip podatka	Opis varijable
id_dvorana	BIGINT (Primarni i strani ključ)	Identifikator dvorane
id_kategorija	BIGINT (Primarni i strani ključ)	Identifikator kategorije

18.1.4 Kategorija

18.1.4.1 Sadrži podatke o svim kategorijama (namjenama)

Atribut	Tip podatka	Opis varijable
id_kategorija	BIGINT (Primarni ključ)	Jedinstveni identifikator kategorije
naziv_kategorija	VARCHAR(255)	Naziv kategorije(namjene) dvorane

18.1.5 Termin

18.1.5.1 Sadrži podatke o svim zauzetim terminima

Atribut	Tip podatka	Opis varijable
id	BIGINT (Primarni ključ)	Jedinstveni identifikator termina
datum_vrijeme_start	TIMESTAMP(6) without time zone	Datum i vrijeme početka termina
id_dvorana	BIGINT (Strani ključ)	Identifikator dvorane na koju se termin odnosi
datum_vrijeme_end	TIMESTAMP(6) without time zone	Datum i vrijeme kraja termina
je_javni_event	INTEGER	Oznaka koja označava je li event privatan ili javan
ime_događanja	VARCHAR(255)	Ime događanja za koje je termin rezerviran(ako je javno)
opis_događanja	TEXT	Opis događanja za koje je termin rezerviran
id_korisnik	BIGINT (Strani ključ)	Identifikator korisnika koji je rezervirao termin

18.1.6 Adresa

18.1.6.1 Sadrži podatke o svim adresama

Atribut	Tip podatka	Opis varijable
id_adresa	BIGINT (Primarni ključ)	Jedinstveni identifikator adrese dvorane
longitude	DOUBLE	Zemljopisna dužina adrese
latitude	DOUBLE	Zemljopisna širina adrese
ulica	VARCHAR(255)	Ulica u kojoj se nalazi dvorana
kucni_broj	VARCHAR(255)	Kućni broj na kojem se nalazi dvorana
id_mjesto	BIGINT (Strani ključ)	Identifikator mjesta u kojem se nalazi dvorana

18.1.7 Mjesto

18.1.7.1 Sadrži podatke o svim mjestima

Atribut	Tip podatka	Opis varijable
id_mjesto	BIGINT (Primarni ključ)	Jedinstveni identifikator mjesta
postanski_broj	BIGINT	Poštanski broj mjesta
naziv_mjesto	VARCHAR(255)	Naziv mjesta

18.1.8 Zahtjev_iznajmljivac

18.1.8.1 Sadrži podatke o svim zahtjevima za promjenu uloge u iznajmljivača

Atribut	Tip podatka	Opis varijable
id	BIGINT (Primarni ključ)	Jedinstveni identifikator zahtjeva
user_id	BIGINT (Strani ključ)	Identifikator korisnika koji želi postati iznajmljivač

18.1.9 Zahtjev_oglas

18.1.9.1 Sadrži podatke o svim zahtjevima za oglašavanje novih dvorana

Atribut	Tip podatka	Opis varijable
id_zajtjev_oglas	BIGINT (Primarni ključ)	Jedinstveni identifikator oglasa dvorane
naz_dvorana	VARCHAR(255)	Naziv dvorane koja se želi oglasiti
kapacitet	INTEGER	Kapacitet dvorane
opis	TEXT	Opis dvorane
cijena_po_satu	REAL	Cijena najma dvorane za jedan sat
days_open	VARCHAR(255)	Podatci o radnim satima za svaki dan u tjednu
street	VARCHAR(255)	Ulica u kojoj se nalazi dvorana
street_number	VARCHAR(255)	Kućni broj na kojem se nalazi dvorana
longitude	DOUBLE	Zemljopisna dužina adrese
latitude	DOUBLE	Zemljopisna širina adrese
postal_code	BIGINT	Poštanski broj grada u kojem se nalazi dvorana
user_id	BIGINT (Strani ključ)	Identifikator korisnika (iznajmljivača) koji oglašava dvoranu
city	VARCHAR(255)	Naziv grada u kojem se nalazi dvorana

18.1.10 Zajtjev_oglas_kategorija

18.1.10.1 Sadrži podatke o svim kategorijama koje su poslane unutar zajtjeva za novu dvoranu

Atribut	Tip podatka	Opis varijable
id_zajtjev_oglas	BIGINT (Primarni i strani ključ)	Identifikator zajtjeva za oglas dvorane
id_kategorija	BIGINT (Primarni i strani ključ)	Identifikator kategorije zajtjeva

18.1.11 Zahtjev_slika

18.1.11.1 Sadrži podatke o svim slikama koje su poslone unutar zahtjeva za novu dvoranu

Atribut	Tip podatka	Opis varijable
id_slika	BIGINT (Primarni ključ)	Jedinstveni identifikator zahtjeva za sliku
poredak_slike	VARCHAR(255)	Atribut koji određuje poredak slika za dvoranu
url_slika	VARCHAR(255)	Poveznica na sliku dvorane
id_zah_tjev_oglas	BIGINT (Strani ključ)	Identifikator zahtjeva za oglas dvorane

18.1.12 Zahtjev_termin

18.1.12.1 Sadrži podatke o svim zahtjevima za najam termina

Atribut	Tip podatka	Opis varijable
id	BIGINT (Primarni ključ)	Jedinstveni identifikator zahtjeva za termin
datum_vrijeme_start	TIMESTAMP(6) without time zone	Datum i vrijeme početka termina
id_dvorana	BIGINT	Identifikator dvorane na koju se termin odnosi
datum_vrijeme_end	TIMESTAMP(6) without time zone	Datum i vrijeme kraja termina
je_javni_event	INTEGER	Oznaka koja označava je li event privatn ili javan
ime_događanja	VARCHAR(255)	Ime događanja za koje je termin rezerviran(ako je javno)
opis_događanja	TEXT	Opis događanja za koje je termin rezerviran

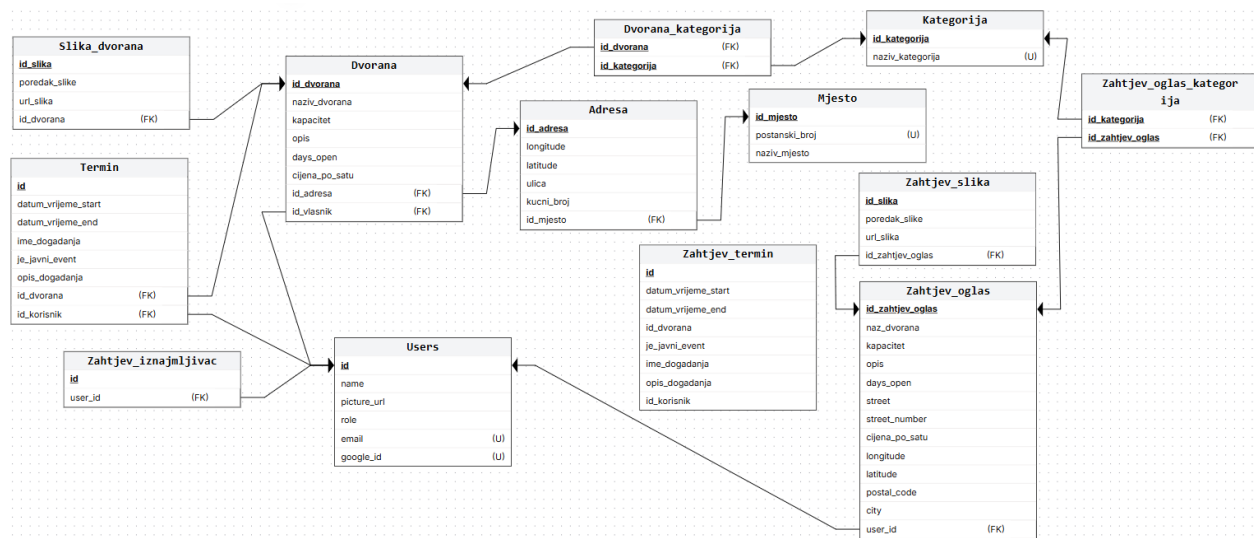


Figure 10: Slika

19 Dijagram razreda

19.1 Cijela Web-Aplikacija



4.1 slika - Dijagram razreda

Aplikacija je strukturirana po slojevitoj Spring Boot arhitekturi i sastoji se od više slojeva. Glavni slojevi su controller, service, repository, model, dto, security, utils, dok glavna klasa DvorankoAPIApplication služi kao ulazna točka aplikacije.

Controller sloj sadrži REST kontrolere koji primaju HTTP zahtjeve s frontenda i prosljeđuju ih servisima na obradu. Svaki kontroler odgovara određenom dijelu poslovne logike, primjerice UserController, DvoranaController ili KategorijaController. Kontroleri koriste Spring anotacije poput @RestController, @RequestMapping, @GetMapping i @PostMapping kako bi definirali API rute. Kada stigne zahtjev, kontroler ga prosljeđuje odgovarajućem servisu. Service sloj sadrži izvršnu logiku aplikacije. Servisi implementiraju glavne funkcionalnosti, kao što su kreiranje korisnika, dohvat dvorana ili obrada zahtjeva, te koriste repository sloj za pristup bazi podataka.

Repository sloj sadrži interfejsne koji nasljeđuju JpaRepository i time omogućuju automatske CRUD operacije nad modelima. Spring automatski implementira ove interfejsne u pozadini, čime se pojednostavljuje pristup podacima u bazi.

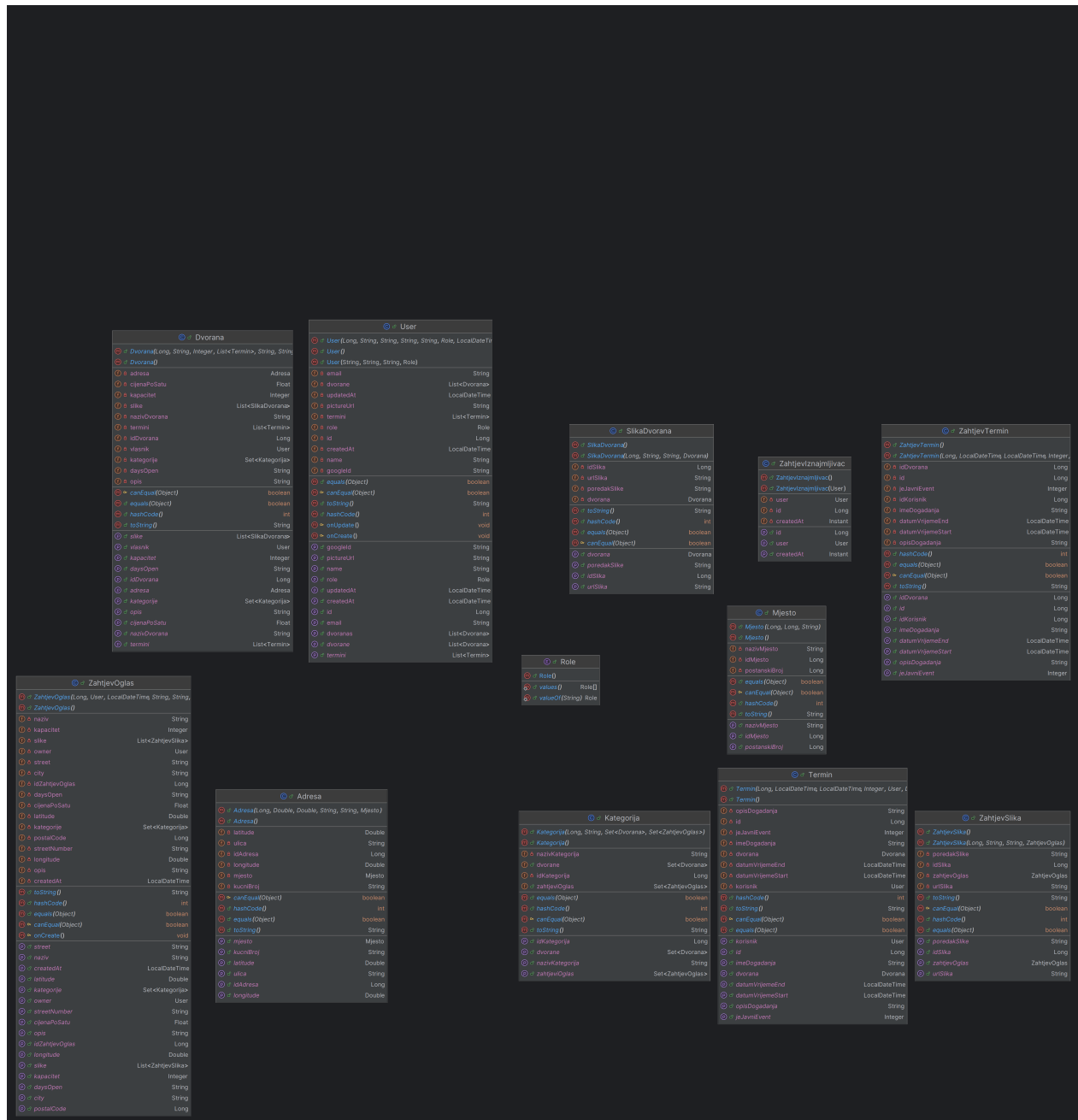
Model sloj sadrži JPA entitete koji predstavljaju tablice u bazi podataka. Svaka klasa je anotirana s `@Entity` i sadrži polja koja odgovaraju kolonama u tablici. Entiteti su međusobno povezani pomoću relacija kao što su `@OneToMany`, `@ManyToOne` i `@ManyToMany`. Među modelima se nalaze `User`, `Role`, `Dvorana`, `Mjesto`, `Kategorija`, `SlikaDvorana`, `ZahtevOglas` i `ZahtevIznajmljivac`, koji zajedno čine osnovu podatkovnog sloja aplikacije.

DTO sloj sadrži objekte za prijenos podataka između slojeva, čime se sprječava izravno izlaganje JPA entiteta prema klijentima. DTO objekti se koriste prilikom primanja podataka iz zahtjeva i slanja odgovora prema frontendu. U podpaketu `createRequest` nalaze se DTO klase koje služe za kreiranje novih entiteta, poput `UserDTO`, `DvoranaDTO`, `KategorijaDTO` i drugih. Kontroleri primaju DTO objekte, prosljeđuju ih servisima, a servisi ih zatim mapiraju na odgovarajuće modele.

Security sloj implementira autentifikaciju i autorizaciju pomoću OAuth2 protokola. Klasa `CustomOAuth2User` predstavlja prilagođeni korisnički model koji dohvaća podatke o korisniku od OAuth providera. `SecurityConfig` konfigurira sigurnosna pravila, definira koje rute zahtijevaju prijavu i upravlja procesom prijave i odjave korisnika.

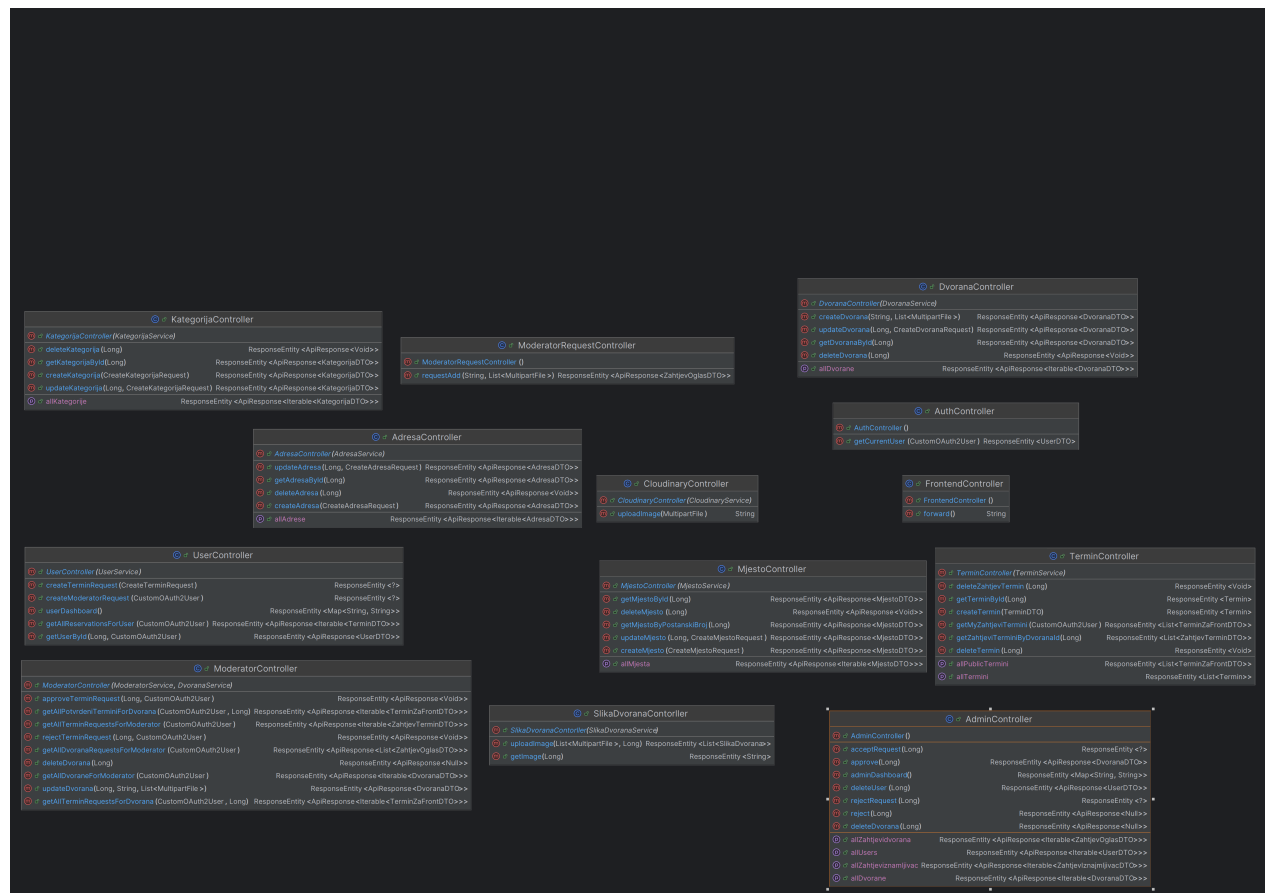
Utils sloj sadrži pomoćne klase koje se koriste u različitim dijelovima projekta. `DtoMapper` služi za konverziju između modela i DTO objekata, dok `ApiResponse` predstavlja generičku strukturu za vraćanje standardiziranih odgovora iz API-ja, uključujući status, poruku i podatke. Glavna klasa `DvorankoAPIApplication` anotirana je s `@SpringBootApplication` i služi kao ulazna točka aplikacije, pokreće Spring aplikaciju i inicijalizira sve beanove potrebne za rad sustava.

19.2 Modeli



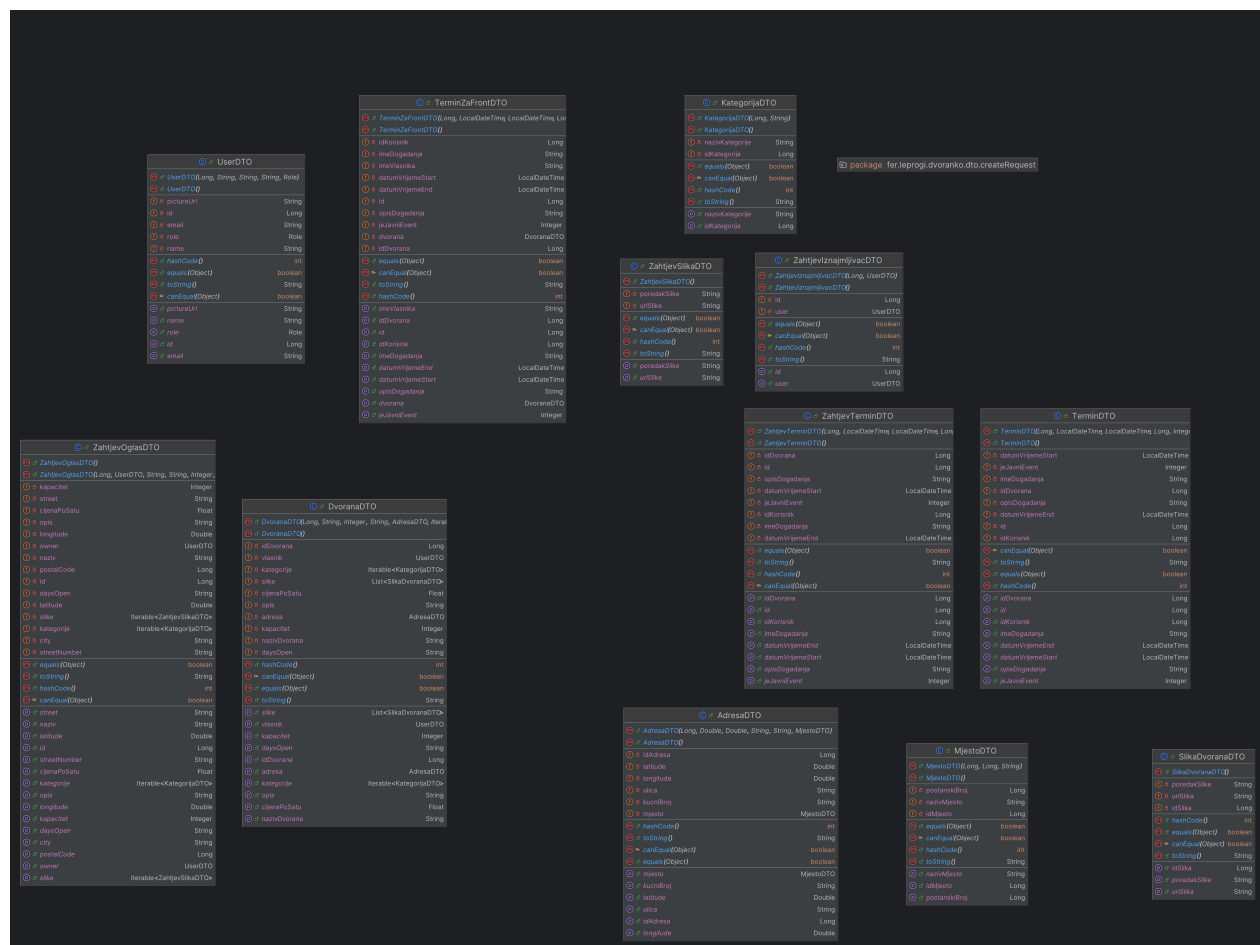
4.2 slika - Dijagram razreda modela

19.3 Kontroleri



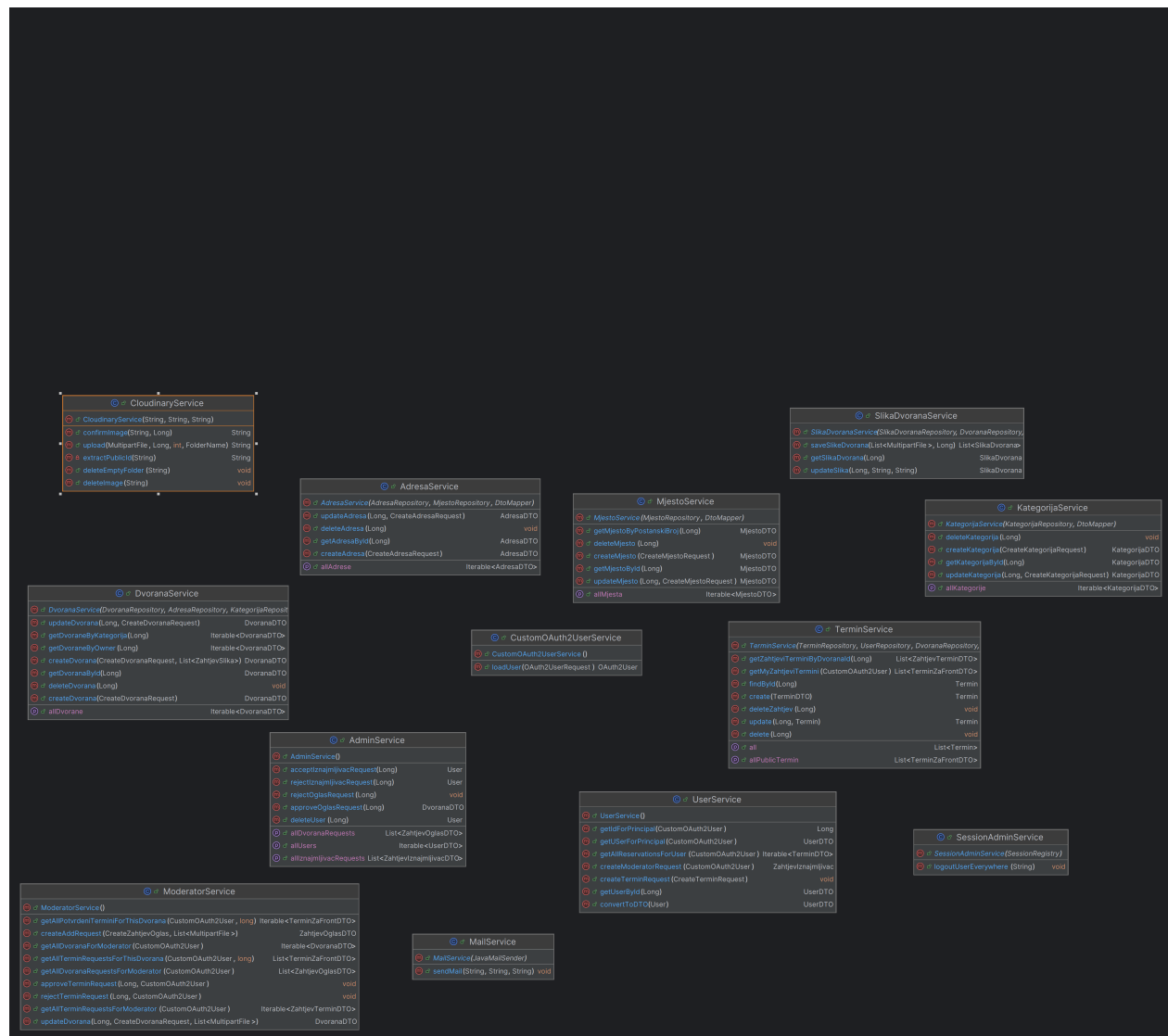
4.3 slika - Dijagram rezreda kontrolera

19.4 DTO



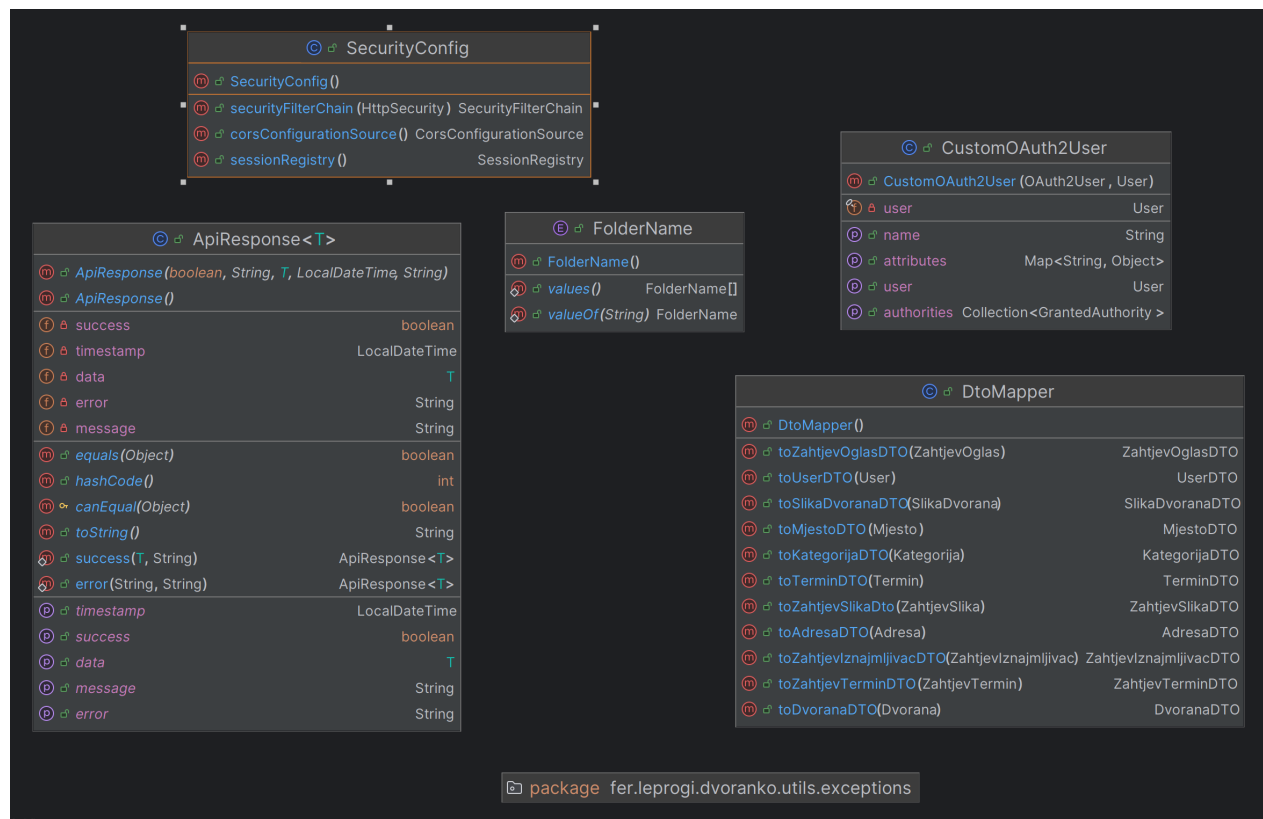
4.4 slika - Dijagram razreda DTO-a

19.5 Servisi



4.5 slika - Dijagram razreda servisi

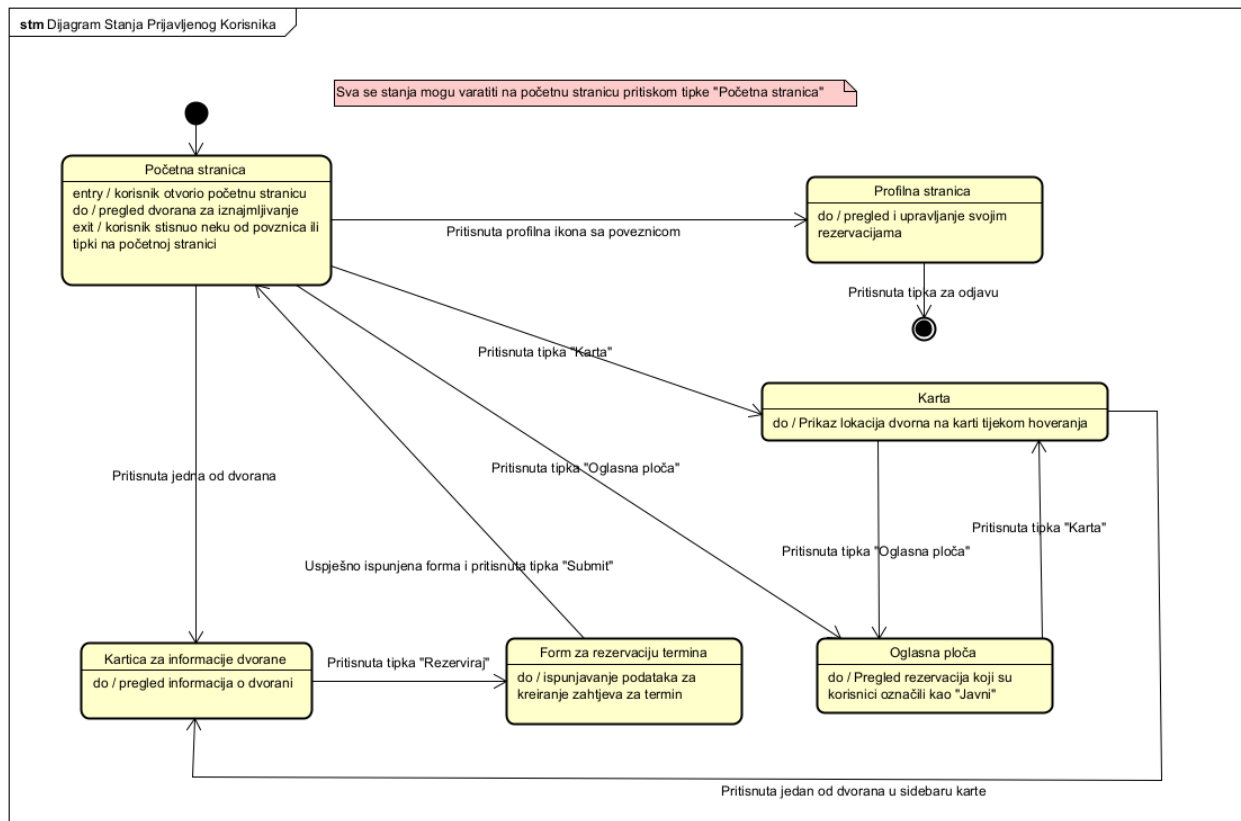
19.6 Sigurnost i pomoćne klase



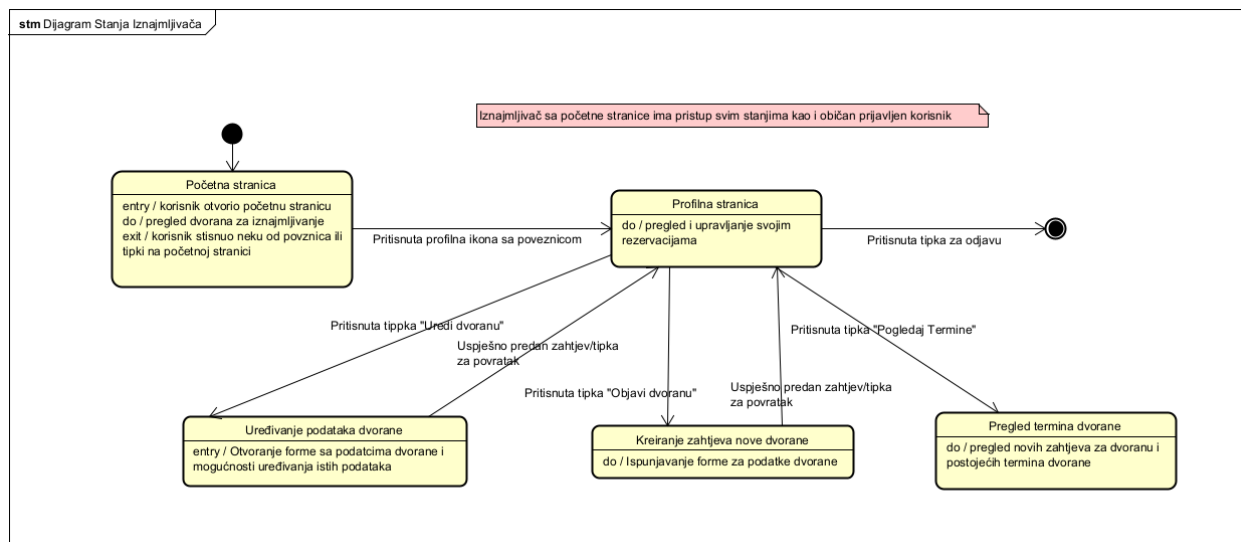
4.6 slika - Dijagram razreda za security i utils

20 Dinamičko ponašanje aplikacije

20.1 UML dijagrami stanja



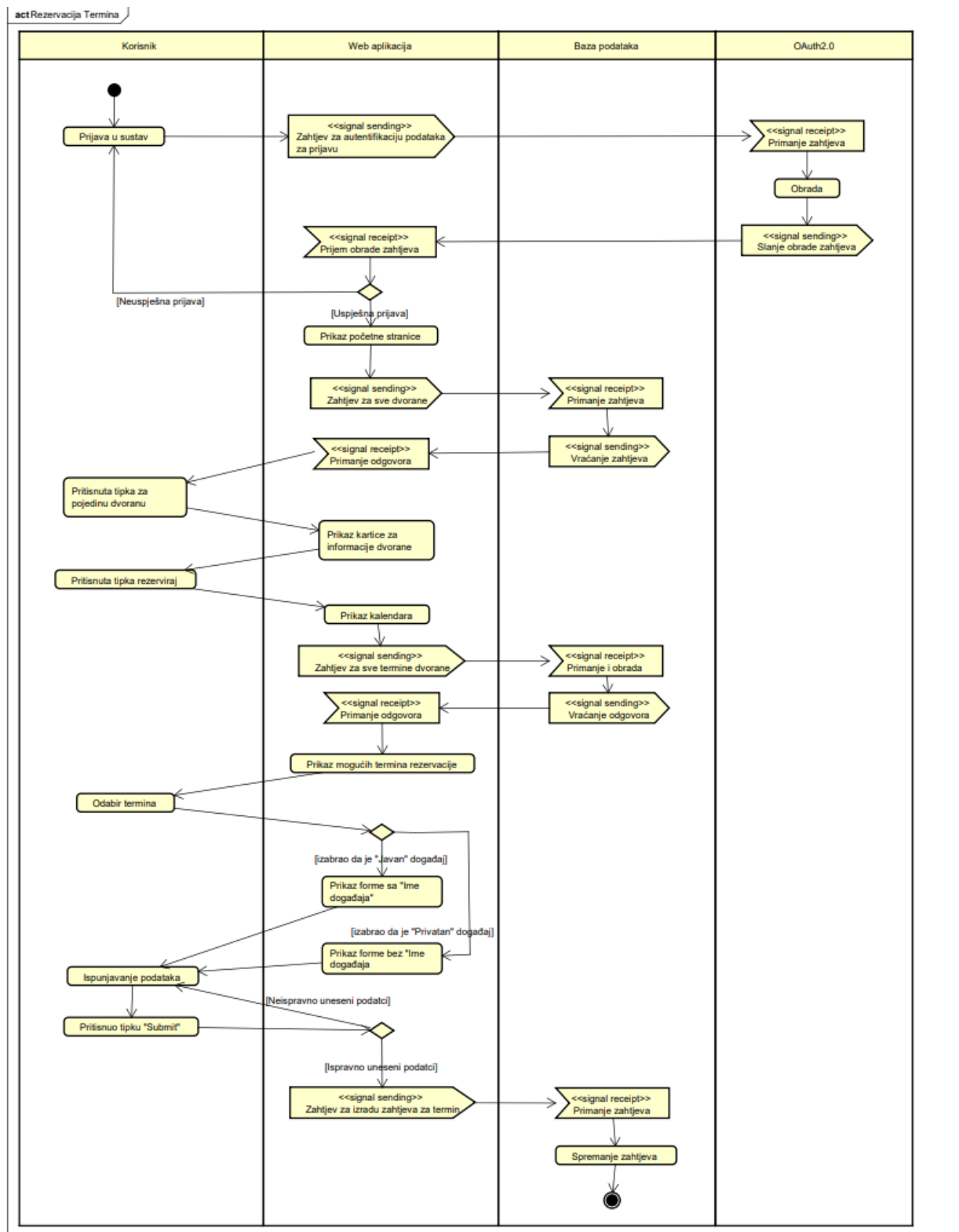
4.7 Slika - Dijagram stanja prijavljenog korisnika *Dijagram stanja prijavljenog korisnika opisuje navigaciju i funkcionalnosti dostupne prijavljenom korisniku aplikacije. Korisnik s početne stranice može pregledavati dvorane, pristupiti karti, oglasnoj ploči ili svojoj profilnoj stranici. Omogućen je pregled detalja odabrane dvorane te pokretanje procesa rezervacije putem obrasca za rezervaciju termina. Korisnik može pregledavati javne rezervacije na oglasnoj ploči i lokacije dvorana na karti. Proces završava odjavom korisnika iz aplikacije.*



4.8. Slika - Dijagram stanja iznajmljivača *Dijagram stanja iznajmljivača prikazuje ponašanje prijavljenog*

korisnika s ulogom iznajmljivača unutar sustava. Početno stanje je početna stranica s koje iznajmljivač može pregledavati dvorane te pristupiti profilnoj stranici. S profilne stranice omogućeno je upravljanje vlastitim dvoranama, uključujući uređivanje postojećih podataka i kreiranje zahtjeva za objavu nove dvorane. Iznajmljivač također može pregledavati termine svojih dvorana. Proces završava odjavom korisnika iz sustava.

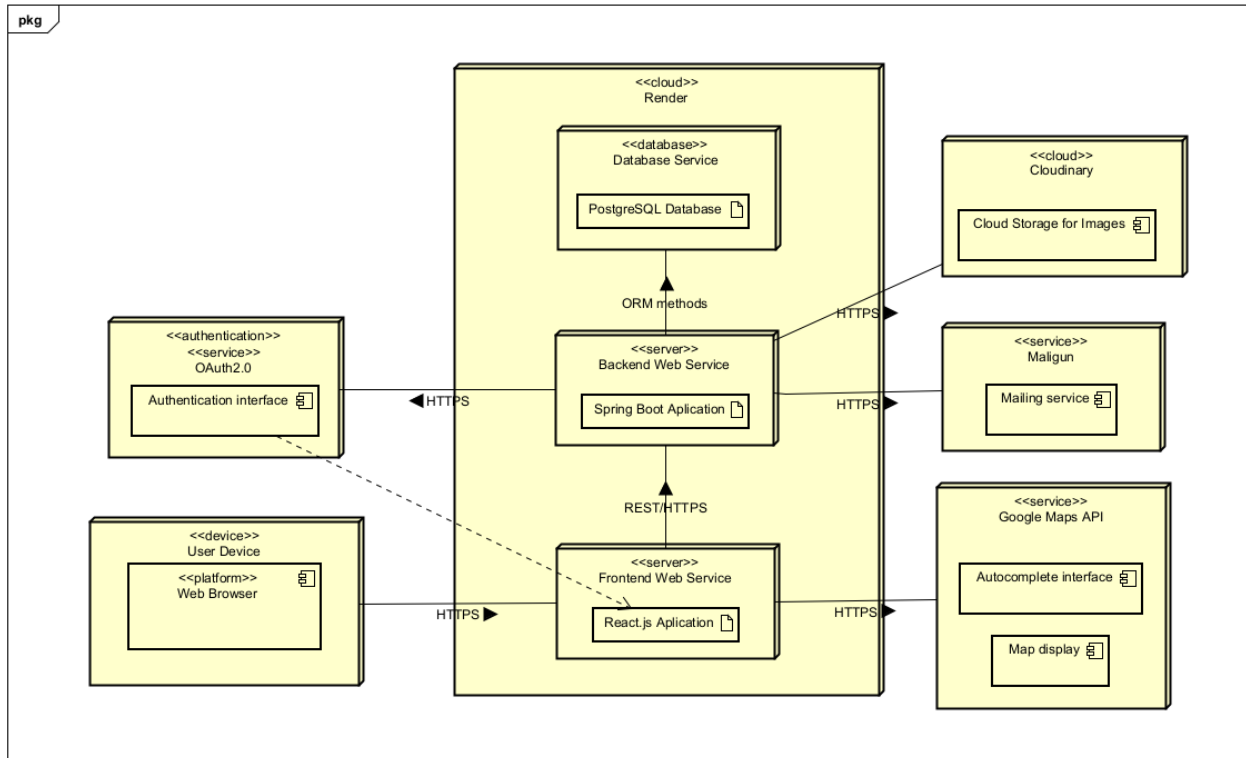
20.2 UML dijagrami aktivnosti



4.9 slika - Dijagram aktivnosti rezervacije termina dvorane Ova UML dijagram aktivnosti prikazuje proces rezervacije termina u web aplikaciji kroz interakciju između korisnika, web aplikacije, baze podataka i OAuth 2.0 sustava. Proces započinje prijavom korisnika u sustav, pri čemu web aplikacija šalje zahtjev za

autentifikaciju prema OAuth 2.0 servisu i obrađuje povratni odgovor. Nakon uspješne prijave, korisniku se prikazuje početna stranica te se dohvaćaju podaci o dostupnim dvoranama iz baze podataka. Korisnik zatim odabire željenu dvoranu, pregledava informacije i kalendar s dostupnim terminima te bira termin za rezervaciju. Ovisno o vrsti događaja (javni ili privatni), korisniku se prikazuje odgovarajuća forma za unos podataka. Nakon validacije unosa, web aplikacija šalje zahtjev za izradu rezervacije prema bazi podataka. Proces završava spremanjem zahtjeva u bazu podataka i uspješnim dovršetkom aktivnosti.

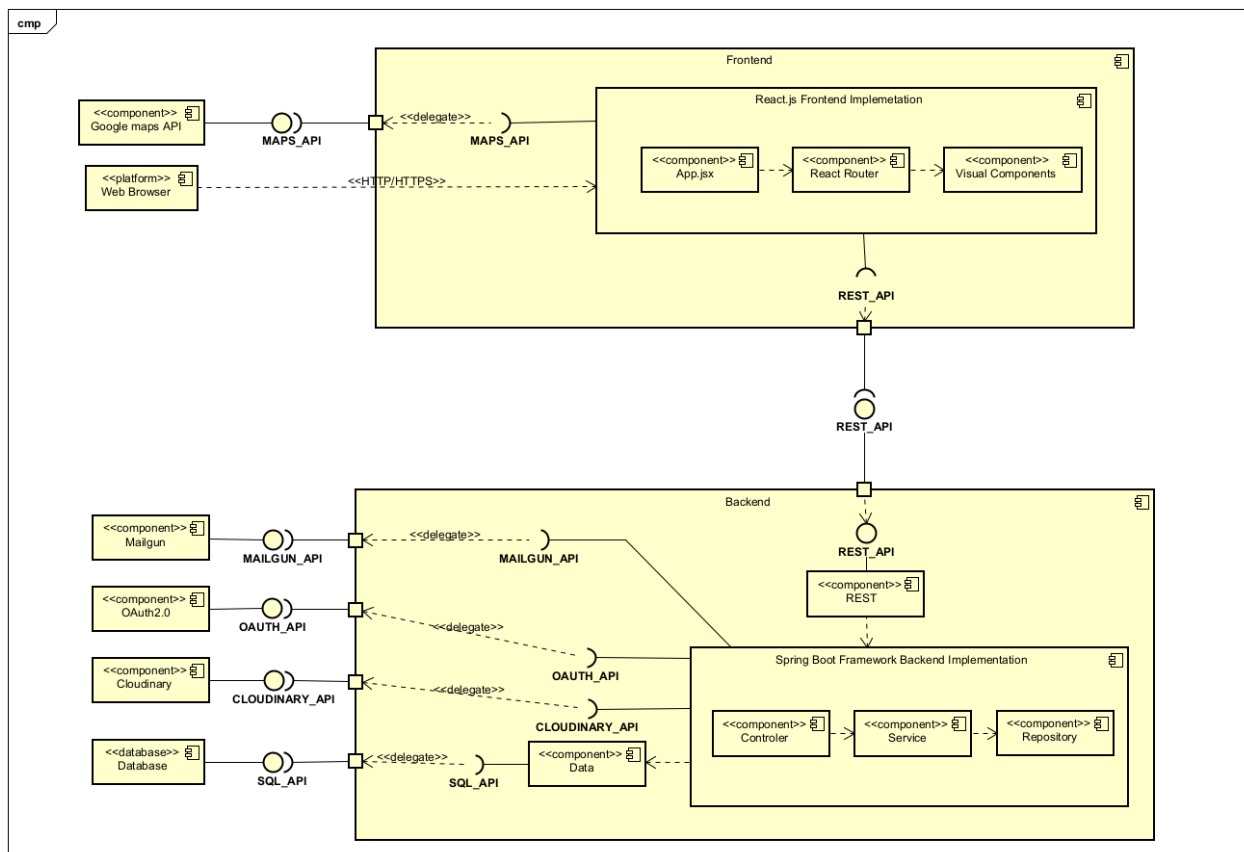
20.3 Dijagram komponentata



Slika 5.1 - Dijagram komponenti

Ovaj dijagram prikazuje ukupnu arhitekturu sustava i način komunikacije između glavnih dijelova aplikacije. Korisnik pristupa sustavu putem web preglednika, koji komunicira s frontend servisom implementiranim u Reactu. Frontend zatim putem REST/HTTPS protokola šalje zahtjeve backend web servisu razvijenom u Spring Bootu. Backend je povezan s PostgreSQL bazom podataka za upravljanje podacima i koristi ORM metode za pristup bazi. Također se integrira s vanjskim servisima kao što su OAuth2 za autentifikaciju, Cloudinary za pohranu slika, Google Maps API za kartografske funkcionalnosti i Mailgun servis za slanje e-mailova

20.4 Dijagram razmjesta



Slika 5.2 - Dijagram razmjesta

Ovaj dijagram prikazuje unutarnju strukturu sustava i odnose između softverskih komponenti. Frontend dio sastoji se od React aplikacije s komponentama App, React Router i vizualnim komponentama koje koriste REST API i Google Maps API. Komunikacija s backendom ostvaruje se preko definiranog REST sučelja. Backend je organiziran u Spring Boot okviru kroz slojeve Controller, Service i Repository, što omogućuje jasnu podjelu odgovornosti. Osim toga, backend koristi posebna sučelja za komunikaciju s OAuth2 servisom, Cloudinary servisom, Mailgun servisom i SQL bazom podataka.

Ispitivanje komponenti 1. Ispitni slučaj

Opis testiranja: U ovom testiranju testirat ćemo radi li kreiranje objekta User dobro i rade li njegovi getteri i setteri kako trebaju, tj. hoće li atributi biti pravilno postavljeni.

Ulazni podatci: Stvorili smo Usera user i pomoću settera postavili smo mu vrijednosti nekih atributa (ID, email, name, pictureURL, googleID, createdAt, updatedAt i Role). Atributi pictureURL i googleID su string pa nismo morali koristiti stvarni URL i googleID, već samo bilo koji string jer ne testiramo funkcionalnost hvatanja URL-ova slike ili googleID-a od vanjskih servisa, već samo postavljanje atributa.

Očekivani rezultat: Gettanjem atributa usera trebali bismo dobiti točno one vrijednosti koje smo unijeli sa setterima.

Dobiveni rezultat: Usporedili smo vrijednosti atributa usera dobivenim getterima i točne one vrijednosti koje smo unijeli na početku te se vrijednosti podudaraju i test prolazi.

Slika 6.1.1 Testiranje postavljanja i dohvaćanja atributa klase User

2. Ispitni slučaj

Opis testiranja: U ovom testiranju testirat ćemo jesu li enumi u ovome projektu dobro postavljeni. Imamo dvije enum klase: Role koja se sastoji od vrijednosti: "USER", "MODERATOR" i "ADMIN" te klasu

```
11 ✓ public class UserTest {  🧑 Matija
12
13     @Test  🧑 Matija
14     void createUserTestAllGood() {
15         ⚡ User user = new User();
16         user.setId(67L);
17         user.setEmail("testiranjeemail123@gmail.com");
18         user.setName("Dvoranko Dvorankić");
19         user.setPictureUrl("link_na_sliku");
20         user.setGoogleId("test_google_id123");
21         LocalDateTime now = LocalDateTime.now(); //želim isto vrijeme za CreatedAt i UpdatedAt
22         user.setCreatedAt(now);
23         user.setUpdatedAt(now);
24         user.setRole(Role.USER);
25
26         assertEquals( expected: 67L, user.getId());
27         assertEquals( expected: "testiranjeemail123@gmail.com", user.getEmail());
28         assertEquals( expected: "Dvoranko Dvorankić", user.getName());
29         assertEquals( expected: "link_na_sliku", user.getPictureUrl());
30         assertEquals( expected: "test_google_id123", user.getGoogleId());
31         assertEquals(now, user.getCreatedAt());
32         assertEquals(now, user.getUpdatedAt());
33         assertEquals(Role.USER, user.getRole());
34     }
35 }
```

est x

73 ms

73 ms

✓ 1 test passed 1 test total, 73 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...

Figure 11: Slika

FolderName koja se sastoji od vrijednosti “zahtjevi” i “dvorane”.

Ulazni podatci: Kao ulazne podatke koristili smo vrijednosti spomenutih enuma.

Očekivani rezultat: Enumi bi trebali biti dobro postavljeni, tj. trebali bismo dobiti očekivane vrijednosti kada ih dohvaćamo.

Dobiveni rezultat: Usporedili smo vrijednosti enuma dobivenim metodom name() s očekivanim vrijednostima. Testovi prolaze.

```
public class EnumTest {  @ Matija

    @Test  @ Matija
    @AutoConfigureTestDatabase
    void EnumTestRoles() {
        assertEquals( expected: "USER", Role.USER.name());
        assertEquals( expected: "MODERATOR", Role.MODERATOR.name());
        assertEquals( expected: "ADMIN", Role.ADMIN.name());
    }

    @Test  @ Matija
    @AutoConfigureTestDatabase
    void EnumTestFolderName() {
        assertEquals( expected: "zahtjevi", FolderName.zahtjevi.name());
        assertEquals( expected: "dvorane", FolderName.dvorane.name());
    }
}

✓ 2 tests passed  2 tests total, 33 ms
```

Figure 12: Slika

Slika 6.1.2 Testiranje dohvaćanja vrijednosti enuma Role i FolderName

3. Ispitni slučaj

Opis testiranja: U ovom testiranju testirat ćemo funkciju createMjesto(CreateMjestoRequest request) iz servisa mjestoService, tj. kako se ponaša kada pokušamo kreirati mjesto koje ne postoji, i kada pokušamo kreirati mjesto koje postoji. Zbog toga što funkcija createMjesto koristi dodatne funkcije kao što su mjestoRepository.existsByPostanskiBroj, mjestoRepository.save i dtoMapper.toMjestoDTO, pretpostavljamo da

te funkcije točno rade te njihove rezultate sami određujemo korištenjem `when()`, zato što želimo testirati izolirano `createMjesto` funkciju.

Ulazni podatci: Za ovo testiranje napravili smo dva testa: u prvom testu kreiramo mjesto i samo jednom zovemo funkciju `createMjesto`, a u drugom kreiramo mjesto i dva puta zaredom zovemo funkciju `createMjesto` na istoj instanci mjesta.

Očekivani rezultat: U prvom testu funkcija bi trebala biti odradena bez bacanja exceptiona, a u drugom testu prvo se treba odraditi bez bacanja exceptiona, a drugim pozivom treba baciti `IllegalArgumentException` jer pokušavamo dodati već postojeće mjesto.

Dobiveni rezultat: Naša pretpostavka bila je dobra, oba testa prolaze.

```
@Test new *
void createMjestoKreiranjeNepostojecegMjesta(){
    CreateMjestoRequest request = new CreateMjestoRequest();
    request.setPostanskiBroj(10000L);
    request.setNazivMjesto("Zagreb");

    when(mjestoRepository.existsByPostanskiBroj(10000L)).thenReturn(true);

    Mjesto savedMjesto = new Mjesto();
    savedMjesto.setPostanskiBroj(10000L);
    savedMjesto.setNazivMjesto("Zagreb");

    when(mjestoRepository.save(any(Mjesto.class))).thenReturn(savedMjesto);

    MjestoDTO dto = new MjestoDTO();
    dto.setPostanskiBroj(10000L);
    dto.setNazivMjesto("Zagreb");
    when(dtoMapper.toMjestoDTO(savedMjesto)).thenReturn(dto);

    assertDoesNotThrow(() -> mjestoService.createMjesto(request));
}
```

Figure 13: Slika

Slika 6.1.3.1 Testiranje kreiranja nepostojeće dvorane

Slika 6.1.3.2 Testiranje kreiranja nepostojeće dvorane, pa onda postojeće

4. Ispitni slučaj

Opis testiranja: U ovom testiranju testiramo funkcionalnost funkcije `getDvoranaById(Long id)`, tj. želimo vidjeti hoće li nam ta funkcija pravilno vratiti dvoranu prema ID-u dvorane, tj. baciti exception ako ta dvorana ne postoji. Kao u 3. ispitnom slučaju, morali smo prepostaviti da funkcije `dvoranaRepository.findById` i `dtoMapper.toDvoranaDTO` rade jer želimo izolirano promatrati funkciju `getDvoranaById`.

Ulazni podatci: Za ovo testiranje u prvom slučaju napravili smo dvoranu te smo ju kreirali i simulirali njeno postojanje u bazi, a u drugom testiranju koristimo izmišljeni ID dvorane koja ne postoji.

```

@Test
void createMjestoKreiranjeIstogMjestaDvaPut() {
    CreateMjestoRequest request = new CreateMjestoRequest();
    request.setPostanskiBroj(10000L);
    request.setNazivMjesto("Zagreb");

    when(mjestoRepository.existsByPostanskiBroj(10000L))
        .thenReturn(true)
        .thenReturn(false);

    Mjesto savedMjesto = new Mjesto();
    savedMjesto.setPostanskiBroj(10000L);
    savedMjesto.setNazivMjesto("Zagreb");

    when(mjestoRepository.save(any(Mjesto.class))).thenReturn(savedMjesto);

    MjestoDTO dto = new MjestoDTO();
    dto.setPostanskiBroj(10000L);
    dto.setNazivMjesto("Zagreb");
    when(dtoMapper.toMjestoDTO(savedMjesto)).thenReturn(dto);

    assertDoesNotThrow(() -> mjestoService.createMjesto(request)); //prvo treba proć
    assertThrows(IllegalArgumentException.class, () -> mjestoService.createMjesto(request)); //
}

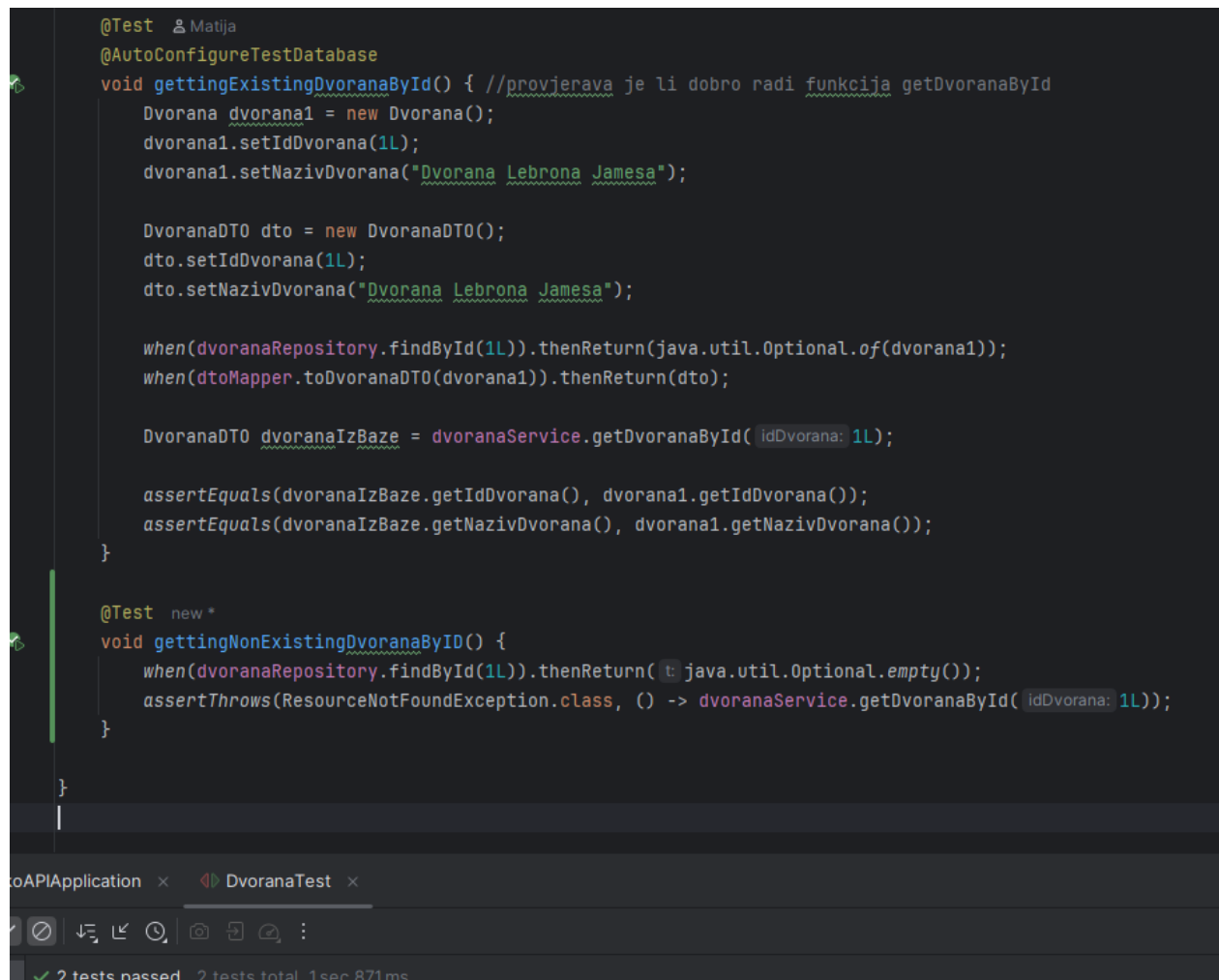
```

✓ 2 tests passed 2 tests total, 1 sec 538 ms

Figure 14: Slika

Očekivani rezultati: U prvom testu ID i naziv uspoređenih dvorana trebao bi biti jednak, a u drugom slučaju trebali bismo dobiti exception jer provjeravamo ID dvorane koji ne postoji.

Dobiveni rezultat: Dobili smo očekivani rezultat, oba testa prolaze.



```
@Test
@AutoConfigureTestDatabase
void gettingExistingDvoranaById() { //provjerava je li dobro radi funkcija getDvoranaById
    Dvorana dvorana1 = new Dvorana();
    dvorana1.setIdDvorana(1L);
    dvorana1.setNazivDvorana("Dvorana Lebrona Jamesa");

    DvoranaDTO dto = new DvoranaDTO();
    dto.setIdDvorana(1L);
    dto.setNazivDvorana("Dvorana Lebrona Jamesa");

    when(dvoranaRepository.findById(1L)).thenReturn(java.util.Optional.of(dvorana1));
    when(dtoMapper.toDvoranaDTO(dvorana1)).thenReturn(dto);

    DvoranaDTO dvoranaIzBaze = dvoranaService.getDvoranaById(idDvorana: 1L);

    assertEquals(dvoranaIzBaze.getIdDvorana(), dvorana1.getIdDvorana());
    assertEquals(dvoranaIzBaze.getNazivDvorana(), dvorana1.getNazivDvorana());
}

@Test
void gettingNonExistingDvoranaById() {
    when(dvoranaRepository.findById(1L)).thenReturn(java.util.Optional.empty());
    assertThrows(ResourceNotFoundException.class, () -> dvoranaService.getDvoranaById(idDvorana: 1L));
}

}
```

2 tests passed 2 tests total, 1sec 871ms

Figure 15: Slika

Slika 6.1.4 Testiranje funkcije getDvoranaById iz servisa DvoranaService

5. Ispitni slučaj

Opis testiranja: U ovom testiranju testiramo funkcionalnost API poziva vezanih za AdresaController. Testiranje provodimo u dva slučaja, u prvom slučaju zovemo točan i postojeći API poziv za dohvaćanje svih adresa, a u drugom slučaju zovemo točan API poziv za dohvaćanje adrese preko ID-a, ali s nepostojećim resursom.

Ulazni podatci: U prvom testu zovemo “/api/public/adrese”, a u drugom testu zovemo “/api/public/adrese/1”.

Očekivani rezultat: U prvom testu trebali bismo dobiti neku listu adresa te dobiti poruku 200OK, a u drugom slučaju trebali bismo dobiti 404 NOT FOUND jer pravilno zovemo API poziv za dohvaćanje neke adrese, ali adresa s tim ID-om ne postoji.

Dobiveni rezultat: Oba testa prolaze, oba dobivena rezultata su očekivana.

Slika 6.1.5 Testiranje dva API poziva iz AdresaController


```
@Test  @ Matija
void correctAPICall() throws Exception {
    AdresaDTO adresaDTO = new AdresaDTO();

    when(adresaService.getAllAdrese()).thenReturn(List.of(adresaDTO));

    mockMvc.perform(get(uriTemplate: "/api/public/adrese"))
        .andExpect(status().isOk())
        .andExpect(content().contentType("application/json"))
        .andDo(print());
}

@Test  @ Matija
void existingAPICallButNotFound() throws Exception {
    when(adresaService.getAdresaById(idAdresa: 1L)).thenReturn(new ResourceNotFoundException("Adresa Not Found"));

    mockMvc.perform(get(uriTemplate: "/api/public/adrese/1"))
        .andExpect(status().isNotFound())
        .andDo(print());
}

}

✓ 2 tests passed 2 tests total, 243 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
```

Figure 16: Slika

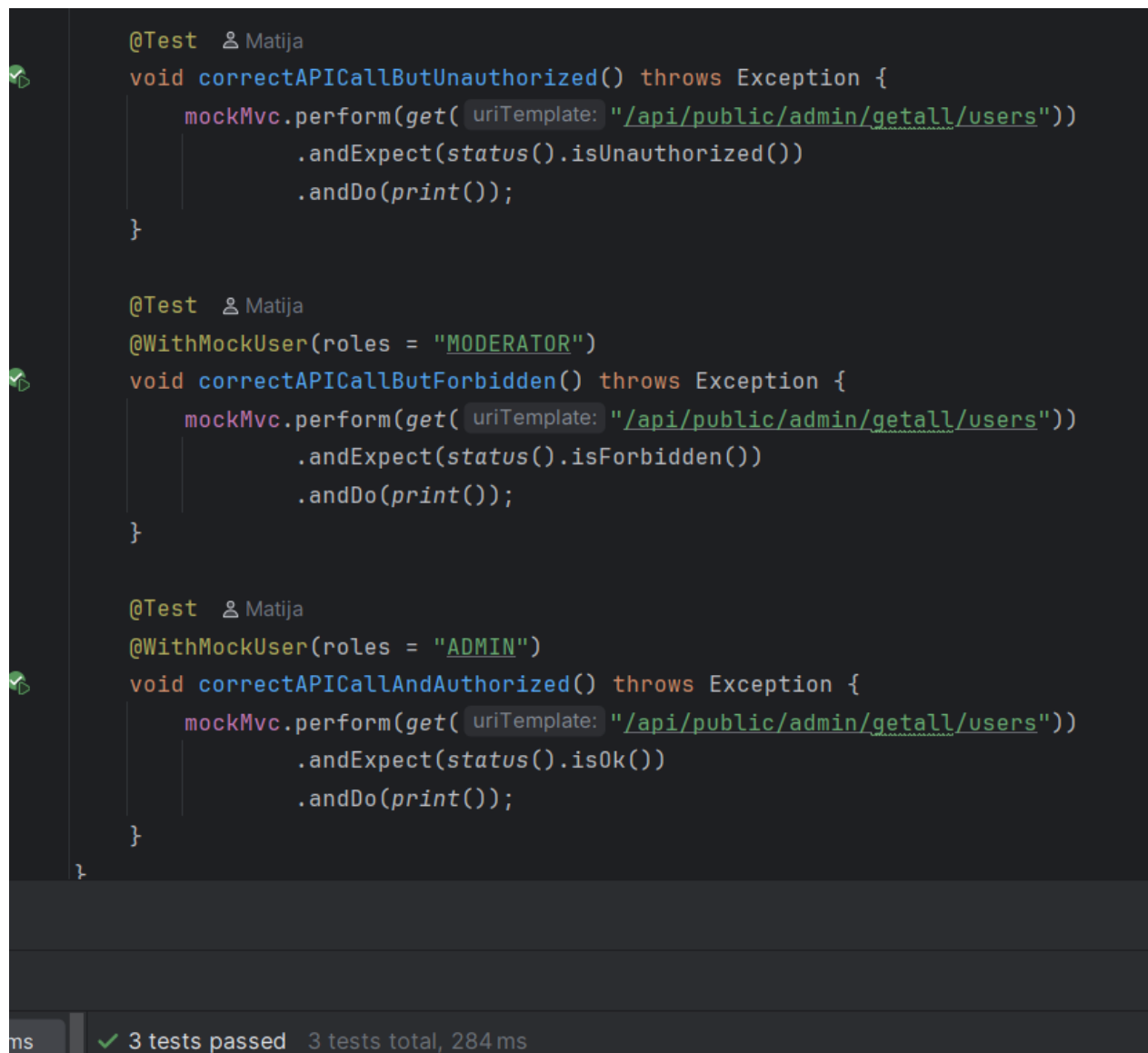
6. Ispitni slučaj

Opis testiranja: U ovom testiranju testiramo adminske API pozive koji bi trebali biti zaštićeni od korisnika koji nisu admini ili koji nisu uopće autorizirani. Zvat ćemo poziv “/api/public/admin/getall/users” koji vraća listu svih Usera i svih njihovih informacija.

Ulazni podatci: Ovo testiranje izvodit ćemo u tri testiranja. U prvom testu zovemo navedeni API poziv, ali kao nelogirani korisnik. U drugom primjeru zovemo API poziv kao logirani MODERATOR, a u trećem primjeru zovemo API poziv kao logirani admin.

Očekivani rezultati: Kad zovemo poziv kao nelogirani korisnik trebali bismo dobiti 401 UNAUTHORIZED, kad zovemo kao moderator trebali bismo dobiti 403 FORBIDDEN, a kad zovemo kao admin trebali bismo dobiti 200 OK.

Dobiveni rezultati: Dobiveni rezultati su očekivani, sva tri testa prolaze.



```
@Test
void correctAPICallButUnauthorized() throws Exception {
    mockMvc.perform(get(uriTemplate: "/api/public/admin/getall/users"))
        .andExpect(status().isUnauthorized())
        .andDo(print());
}

@Test
@WithMockUser(roles = "MODERATOR")
void correctAPICallButForbidden() throws Exception {
    mockMvc.perform(get(uriTemplate: "/api/public/admin/getall/users"))
        .andExpect(status().isForbidden())
        .andDo(print());
}

@Test
@WithMockUser(roles = "ADMIN")
void correctAPICallAndAuthorized() throws Exception {
    mockMvc.perform(get(uriTemplate: "/api/public/admin/getall/users"))
        .andExpect(status().isOk())
        .andDo(print());
}
```

ms ✓ 3 tests passed 3 tests total, 284 ms

Figure 17: Slika

Slika: 6.1.6 testiranje Admin API poziva

21 Ispitivanje sustava

Za testiranje sustava korišten je Selenium WebDriver te je kod pisan u javi. Ulogirani korisnik se mora

21.1 1. Ispitni slučaj: Provjera uspješnosti ažuriranja podataka dvorane

Pretpostavka:

Testni user koji se koristi je iznajmljivac (moderator) te u aplikaciji ima potvrđenu dvoranu, odnosno v

Cilj:

Automatsko testiranje ažuriranja već postojeće dvorane. Korisnik svojoj dvorani promijeni ime dvorane,

Koraci ispitivanja:

1. Navigacija na stranicu moj profil
2. Pritisak na gumb za dodavanje dvorane (+) koji otvara form za kreiranje dvorane
3. Ažuriranje podataka na formu
4. Slanje forme za ažuriranje podataka
5. Povratak na stranicu moj profil
6. Provjera jesu li se podatci o dvorani ažurirali na mom profilu

Očekivani rezultat

1. Uspješno navigacija na moj profil
2. Uspješna navigacija na form
3. Uspješno uneseni podatci
4. Uspješno slanje forme za ažuriranje podataka
5. Uspješan povratak na stranicu moj profil
6. Uspješna promjena podataka

Slika: 6.2.1.1 testiranje ažuriranja podataka o dvorani kod

Slika: 6.2.1.2 testiranje ažuriranja podataka o dvorani kod

Slika: 6.2.1.3 rezultati testa za ažuriranje podataka o dvorani

21.2 2. Ispitni slučaj: Provjera uspješnosti ažuriranja podataka dvorane

Pretpostavka:

Testni korisnik koji se koristi je admin te u aplikaciji postoji već potvrđena dvorana.

Cilj:

Automatsko testiranje brisanja već postojeće dvorane od strane admina. Admin obriše bilo koju dvoranu te

Koraci ispitivanja:

1. Navigacija na stranicu za admina
2. Pritisak na gumb za prikaz postojećih dvorana i korisnika
3. Pritisak na gumb za brisanje odabrane dvorane
4. Osvježavanje stranice kako bi se ponovno fetchale objavljene dvorane za provjeru
5. Ponovna navigacija na prikaz postojećih dvorana i korisnika
6. Provjera je li odabrana dvorana obrisana

Očekivani rezultat

1. Uspješna navigacija na stranicu za admina
2. Uspješan prikaz postojećih dvorana i korisnika
3. Uspješan odabir i brisanje dvorane
4. Stranica je osvježena i podaci su ponovno dohvaćeni

```

29 @Test  & Vid Martin *
30 public void testEditDvorana(){
31
32     String newIme = "Testna dvorana (edit test)";
33     int newPrice = 50;
34     int newMaxCapacity = 200;
35     String newOpis = "Opis nove testne dvorane, promijenjen u testu";
36
37
38     WebDriverManager.chromedriver().setup();
39     WebDriver driver = new ChromeDriver();
40     WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
41
42     driver.get("http://localhost:8080/test/login/moderator");
43     driver.get("http://localhost:5173/my-profile");
44
45     System.out.println("Navigating to create dvorana page");
46
47     WebElement editVenueButton = wait.until(
48         ExpectedConditions.visibilityOfElementLocated(By.id("uredi-dvoranu-33"))
49     );
50
51     editVenueButton.click();
52     System.out.println("Clicking edit dvorana button for dvorana with id 33");
53
54
55
56     WebElement name = driver.findElement(By.id("naziv-input"));
57     name.clear();
58     name.sendKeys(newIme);
59     System.out.println("Inputing dvorana name");
60
61     WebElement maxCapacity = driver.findElement(By.id("kapacitet-input"));
62     maxCapacity.clear();
63     maxCapacity.sendKeys(String.valueOf(newMaxCapacity));
64     System.out.println("Inputing max capacity");
65
66     WebElement price = driver.findElement(By.id("cijena-input"));
67     price.clear();
68     price.sendKeys(String.valueOf(newPrice));
69     System.out.println("Inputing price");
70
71     WebElement categoryButton = wait.until(
72         ExpectedConditions.elementToBeClickable(
73             By.xpath(
74                 xpathExpression: "//div[@class='flex flex-wrap gap-[5px] w-[90%]']/button[normalize-space()='Odboraka']"
75             )
76         );
77     categoryButton.click();
78     System.out.println("Clicking category");
79
80     WebElement eopis = driver.findElement(By.id("opis-input"));
81     eopis.clear();
82     eopis.sendKeys(newOpis);
83     System.out.println("Inputing opis");

```

Figure 18: Slika

```

84  > ..
126
127  WebElement submitButton = driver.findElement(By.id("submit-update-button"));
128  submitButton.click();
129  System.out.println("Submitting form");
130
131  System.out.println("Navigating back to profile page");
132
133
134  WebElement visitDvorana = wait.until(ExpectedConditions.elementToBeClickable(By.id("view-venue-33")));
135  visitDvorana.click();
136  System.out.println("Navigating to edited dvorana page");
137
138
139  System.out.println("Verifying edited dvorana data");
140
141  WebElement editedIme = wait.until(ExpectedConditions.elementToBeClickable(By.id("ime-dvorana")));
142  String ime = editedIme.getText();
143
144  WebElement editedKapacitet = driver.findElement(By.id("kapacitet"));
145  int kapacitet = Integer.parseInt(editedKapacitet.getText().replace(target: "Kapacitet: ", replacement: "").trim());
146
147
148  WebElement editedCijena = driver.findElement(By.id("cijena"));
149  int cijena = Integer.parseInt(editedCijena.getText().replace(target: "Cijena po satu: ", replacement: "").replace(target: "€/h", replacement: "").trim());
150
151
152  WebElement editedOpis = driver.findElement(By.id("opis"));
153  String opis = editedOpis.getText().replace(target: "Opis prostora: ", replacement: "");
154
155  assert ime.equals(newIme) : "Ime dvorane nije ispravno azurirano";
156  assert kapacitet == newMaxCapacity : "Kapacitet dvorane nije ispravno azurirano";
157  assert cijena == newPrice : "Cijena dvorane nije ispravno azurirano";
158  assert opis.equals(newOpis) : "Opis dvorane nije ispravno azurirano";
159
160  System.out.println("Edit dvorana test completed successfully");
161
162  driver.quit();
163  }

```

Figure 19: Slika

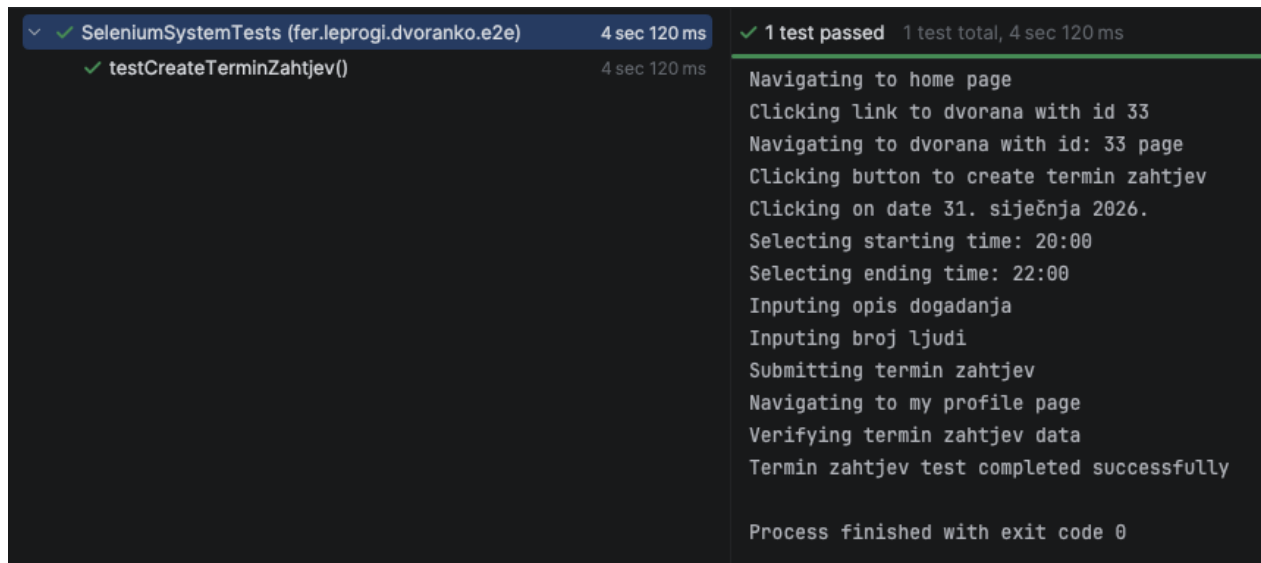


Figure 20: Slika

5. Uspješna navigacija na prikaz postojećih dvorana i korisnika
6. Odabrana dvorana više nije prikazana; potvrđen je njen brisanje

Slika: 6.2.2.1 testiranje ažriranja podataka o dvorani

Slika: 6.2.2.2 rezultati testa za ažuriranje podataka o dvorani

21.3 3. Ispitni slučaj: Provjera uspješnosti kreiranja rezervacije

Pretpostavka:

Testni korisnik koji se koristi je user te u aplikaciji postoji već potvrđena dvorana.

Cilj:

Automatsko testiranje zahtjeva za rezervaciju termina. Korisnik odabere dvoranu te za istu odabere datum

Koraci ispitivanja:

1. Navigacija na početnu stranicu
2. Odabir dvorane i pritisak na nju
3. Pritisak na gumb za rezervaciju termina
4. Odabir slobodnog termina te ispunjavnje potrebnih informacija
5. Pritisak na gumb rezerviraj i slanje zahtjeva
6. Navigacija na stranicu moj profil
7. Provjera postoji li zahtjev za termin pod mojim rezervacijama

Očekivani rezultat

1. Uspješna navigacija na početnu stranicu
2. Uspješan prikaz detalja odabrane dvorane
3. Uspješan prikaz forme za rezervaciju termina
4. Uspješno odabran slobodan termin i uneseni potrebni podatci
5. Uspješno poslan zahtjev za rezervaciju termina
6. Uspješna navigacija na stranicu moj profil
7. Zahtjev za rezervaciju termina je vidljiv pod mojim rezervacijama

Slika: 6.2.3.1 testiranje ažriranja podataka o dvorani

```

167  @Test  & Vid Martin +
168  public void testAdminDeleteDvorana(){
169      WebDriverManager.chromedriver().setup();
170      WebDriver driver = new ChromeDriver();
171      WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
172
173      int dvoranaId = 31;
174
175      driver.get("http://localhost:8080/test/login/admin");
176      driver.get("http://localhost:5173/admin");
177
178      System.out.println("Navigating to admin page");
179
180      driver.findElement(By.xpath(xpathExpression: "//button[text()='Prikaži korisnike i dvorane']")).click();
181      System.out.println("Clicking button to show users and venues");
182
183      wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("delete-dvorana-btn-"+dvoranaId))).click();
184      System.out.println("Clicking delete button for dvorana with id "+dvoranaId);
185
186      wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("delete-dvorana-btn-"+dvoranaId)));
187      System.out.println("Verifying that delete button is no longer visible");
188
189      driver.navigate().refresh();
190      System.out.println("Refreshing the page to confirm deletion, (frontend fetches data again)");
191
192      driver.findElement(By.xpath(xpathExpression: "//button[text()='Prikaži korisnike i dvorane']")).click();
193      System.out.println("Clicking button to show users and venues again");
194
195      List<WebElement> elements = driver.findElements(By.id("delete-dvorana-btn-"+dvoranaId));
196
197      assert elements.isEmpty() :
198      | "Dvorana with id " + dvoranaId + " is not deleted and still visible on admin page";
199
200      System.out.println("Admin delete dvorana test completed successfully");
201
202      driver.quit();
203
204  }
205

```

Figure 21: Slika

✓ SeleniumSystemTests (fer.leprogi.dvoranko.e2e) 3 sec 184 ms ✓ testAdminDeleteDvorana() 3 sec 184 ms	✓ 1 test passed 1 test total, 3 sec 184 ms Navigating to admin page Clicking button to show users and venues Clicking delete button for dvorana with id 31 Verifying that delete button is no longer visible Refreshing the page to confirm deletion, (frontend fetches data again) Clicking button to show users and venues again Admin delete dvorana test completed successfully Process finished with exit code 0
--	---

Figure 22: Slika

```

206 @Test 3 Vid Martin *
207 public void testCreateTerminZahtjev(){
208     WebDriverManager.chromedriver().setup();
209     WebDriver driver = new ChromeDriver();
210     WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
211
212     int dvoranaId = 33;
213     String opisText = "Na ovom terminu radimo testiranje sustava";
214     int brojLjudiText = 10;
215     String odVrijeme = "20:00";
216     String doVrijeme = "22:00";
217     String datum = "31.01.2026.";
218
219     driver.get("http://localhost:8080/test/login/moderator");
220     driver.get("http://localhost:5173/");
221
222     System.out.println("Navigating to home page");
223
224     wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("dvorana-link-" + dvoranaId))).click();
225     System.out.println("Clicking link to dvorana with id " + dvoranaId);
226
227     System.out.println("Navigating to dvorana with id: " + dvoranaId + " page");
228
229     wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("rezerviraj-dvoranu-btn"))).click();
230     System.out.println("Clicking button to create termin zahtjev");
231
232     wait.until(ExpectedConditions.visibilityOfElementLocated(
233         By.xpath("//a[@aria-label='31. siječnja 2026.' and text()='31']"))).click();
234     System.out.println("Clicking on date 31. siječnja 2026.");
235
236     wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("time-select-" + odVrijeme))).click();
237     System.out.println("Selecting starting time: " + odVrijeme);
238
239     wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("time-select-" + doVrijeme))).click();
240     System.out.println("Selecting ending time: " + doVrijeme);
241
242     WebElement opis = wait.until(ExpectedConditions.visibilityOfElementLocated(
243         By.id("input-opis-dogadjanja")));
244     opis.sendKeys(...keysToSend: opisText);
245     System.out.println("Inputing opis dogadjanja");
246
247     WebElement brojLjudi = wait.until(ExpectedConditions.visibilityOfElementLocated(
248         By.id("input-broj-ljudi")));
249     brojLjudi.sendKeys(String.valueOf(brojLjudiText));
250     System.out.println("Inputing broj ljudi");
251
252     wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("submit-zahtjev-termin-btn"))).click();
253     System.out.println("Submitting termin zahtjev");
254
255     // driver.get("http://localhost:5173/my-profile/" + dvoranaId);
256     System.out.println("Navigating to my profile page");
257
258
259     wait.until(ExpectedConditions.presenceOfElementLocated(
260         By.cssSelector("div.flex-1.rounded-xl.p-5.flex.flex-col.gap-3")));
261
262     List<WebElement> kartice = driver.findElements(
263         By.cssSelector("div.flex-1.rounded-xl.p-5.flex.flex-col.gap-3"));
264
265     String opisOut = null;
266     String odOut = null;
267     String doOut = null;
268     String datumOut = null;
269
270     System.out.println("Verifying termin zahtjev data");
271     for (WebElement kartica : kartice) {...}
272
273     assert opisText.equals(opisOut) : "Opis dogadjanja nije ispravan";
274     assert odVrijeme.equals(odOut) : "Vrijeme pocetka nije ispravno";
275     assert doVrijeme.equals(doOut) : "Vrijeme zavrsetka nije ispravno";
276     assert datum.equals(datumOut) : "Datum nije ispravan";
277
278     System.out.println("Termin zahtjev test completed successfully");
279
280     driver.quit();
281 }

```

Figure 23: Slika
60

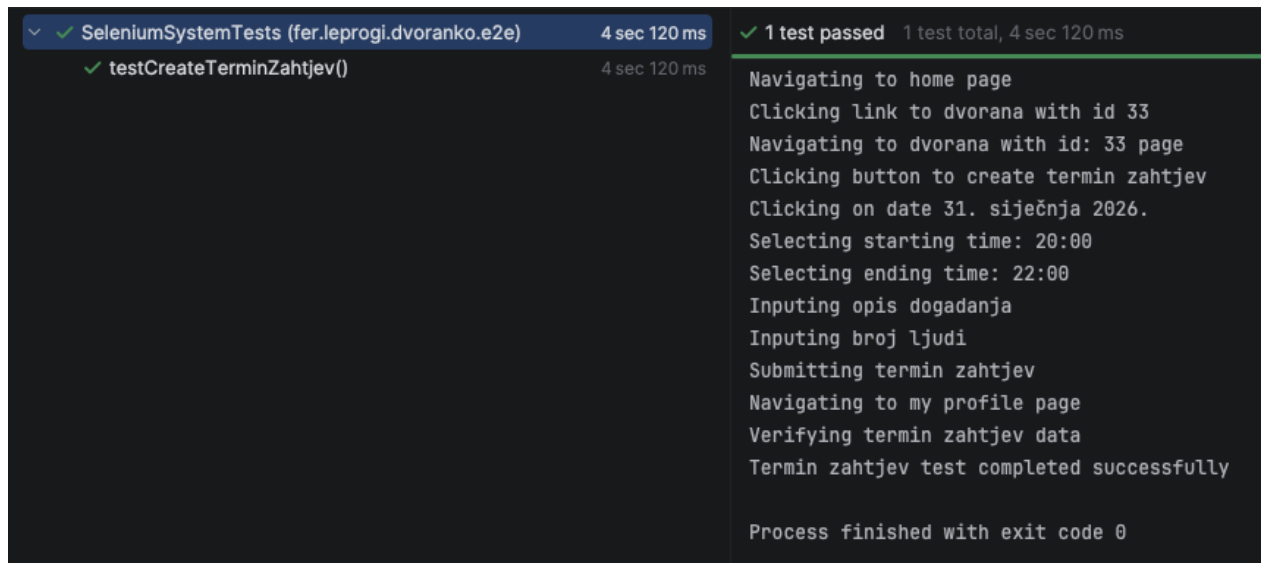


Figure 24: Slika

Slika: 6.2.3.2 rezultati testa za ažuriranje podataka o dvorani

21.4 4. Ispitni slučaj: Provjera uspješnosti filtriranja dvorana

Pretpostavka:

Testni korisnik koji se koristi je user te u aplikaciji postoji već potvrđena dvorana s postavljenom cijenom.

Cilj:

Automatsko testiranje filtriranja dvorani po cijeni. Korisnik utvrdje da postoji dvorana s cijenom u rasponu.

Koraci ispitivanja:

1. Navigacija na početnu stranicu
2. Pritisak na gumb za filter i odabir raspona cijena iz dropdown menia
3. Pritisak na gumb filtriraj
4. Provjera da ne postoji dvorana koja ima cijenu po satu izvan zadanog raspona
5. Pritisak na gumb rezerviraj i slanje zahtjeva
6. Navigacija na stranicu moj profil
7. Provjera postoji li zahtjev za termin pod mojim rezervacijama

Očekivani rezultat

1. Uspješna navigacija na početnu stranicu
2. Uspješan prikaz opcija za filtriranje i odabran raspon cijena
3. Uspješno primijenjen filter cijena
4. Prikazane su samo dvorane čija je cijena po satu unutar zadanog raspona
5. Uspješno poslan zahtjev za rezervaciju termina za odabranu dvoranu
6. Uspješna navigacija na stranicu moj profil
7. Zahtjev za rezervaciju termina vidljiv je pod mojim rezervacijama

Slika: 6.2.4.1 testiranje ažuriranja podataka o dvorani

Slika: 6.2.4.2 rezultati testa za ažuriranje podataka o dvorani

```

310
311 @Test 吕 Vid Martin *
312 public void testFilterDvorana(){
313     WebDriverManager.chromedriver().setup();
314     WebDriver driver = new ChromeDriver();
315     WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
316
317     driver.get("http://localhost:8080/test/login/user");
318     driver.get("http://localhost:5173/");
319
320     String cijenaRange = "40-60";
321
322     System.out.println("Navigating to home page");
323
324     //cekaj da se ucita neka dvorana
325     wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("a[id^='dvorana-link-']")));
326     List<WebElement> kartice = driver.findElements(By.cssSelector("a[id^='dvorana-link-']"));
327
328     boolean postojiDvorana = false;
329     for (WebElement kartica : kartice) {...}
340     assert postojiDvorana : "Nema dvorana sa cijenom iz raspona " + cijenaRange;
341
342     driver.findElement(By.id("filter-btn")).click();
343     System.out.println("Clicking filter button");
344
345     WebElement cijenaOptions = driver.findElement(By.id("filter-cijena-options"));
346     Select cijenaSelect = new Select(cijenaOptions);
347     cijenaSelect.selectByValue(cijenaRange);
348     System.out.println("Selecting new cijena range: " + cijenaRange);
349
350     driver.findElement(By.id("filter-submit-btn")).click();
351     System.out.println("Submitting filter form");
352
353     System.out.println("Verifying filtered dvorana data");
354
355     //cekaj da se ucita neka dvorana
356     wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("a[id^='dvorana-link-']")));
357     List<WebElement> karticeAfter = driver.findElements(By.cssSelector("a[id^='dvorana-link-']"));
358
359     for (WebElement kartica: karticeAfter) {...}
360
361     System.out.println("Filter dvorana test completed successfully, " +
370         "no dvorana found outside the cijena range : " + cijenaRange);
371
372     driver.quit();
373 }
374

```

Figure 25: Slika

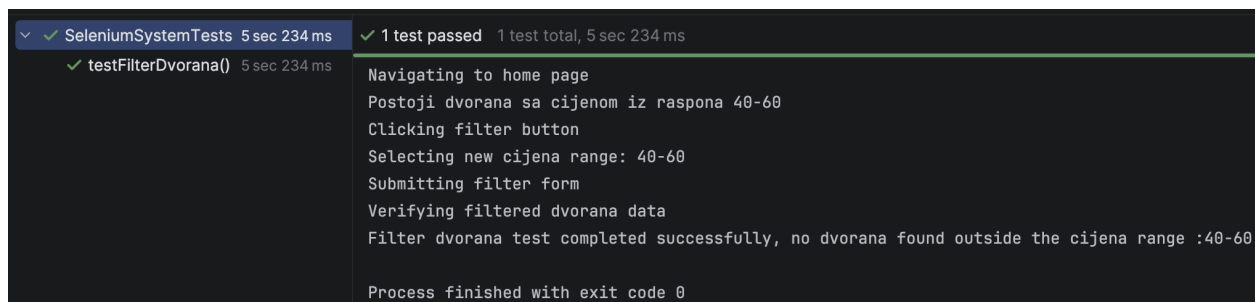


Figure 26: Slika

22 Korištene tehnologije i alati

22.1 Programski jezici

U razvoju aplikacije korišten je programski jezik Java (verzija 21) za implementaciju poslužiteljskog dijela sustava pomoću Spring Boot radnog okvira. Java 21 predstavlja LTS (Long-Term Support) verziju jezika te donosi poboljšanja u performansama, sigurnosti i stabilnosti, što je čini pogodnom za razvoj pouzdanih i skalabilnih serverskih aplikacija. Za razvoj klijentskog dijela aplikacije korišten je JavaScript (ES2023), suvremena verzija jezika koja omogućuje pisanje modularnog i čitljivog koda te učinkovitu izradu interaktivnih korisničkih sučelja pomoću React biblioteke. Struktura i prezentacija korisničkog sučelja izrađene su pomoću HTML5 i CSS3 tehnologija, dok je za stiliziranje korišten Tailwind CSS, koji omogućuje brzo i konzistentno oblikovanje sučelja korištenjem unaprijed definiranih stilskih klasa.

22.2 Radni okviri i biblioteke

Frontend:

Za razvoj klijentskog dijela aplikacije korištena je biblioteka React (verzija 19.1.1), moderna JavaScript biblioteka za izgradnju interaktivnih korisničkih sučelja temeljenih na komponentama. React omogućuje izradu modularnih i ponovno iskoristivih komponenti te učinkovito upravljanje stanjem aplikacije, čime se povećava održivost i skalabilnost koda. Kao alat za razvojno okruženje i izgradnju aplikacije korišten je Vite (verzija 7.1.7), koji omogućuje izuzetno brzo pokretanje razvojnog poslužitelja i optimizirani proces izgradnje aplikacije za produkcijsko okruženje. Za stiliziranje korisničkog sučelja korišten je Tailwind CSS (verzija 3.x), utility-first CSS framework koji omogućuje brzo i konzistentno oblikovanje sučelja izravno unutar JSX koda bez potrebe za pisanjem velike količine vlastitog CSS-a. Vizualni izgled i prototip korisničkog sučelja prethodno su izrađeni pomoću alata Figma, koji je korišten za dizajn, planiranje rasporeda elemenata i usklađivanje izgleda aplikacije prije same implementacije.

Backend:

Poslužiteljski dio aplikacije razvijen je pomoću radnog okvira Spring Boot (verzija 3.5.6), koji omogućuje brzu izradu RESTful servisa, automatsku konfiguraciju aplikacije i jednostavno upravljanje ovisnostima. Spring Boot znatno pojednostavljuje razvoj serverskih aplikacija te omogućuje jasnu organizaciju slojeva aplikacije. Za rad s bazom podataka korišten je Spring Data JPA, koji pruža apstrakciju nad pristupom podacima i omogućuje jednostavno upravljanje perzistentnim entitetima bez potrebe za pisanjem velike količine SQL koda. Kao ORM (Object-Relational Mapping) alat korišten je Hibernate, koji omogućuje mapiranje Java objekata u relacijske tablice baze podataka te pojednostavljuje rad s perzistentnim podacima.

22.3 Baza podataka

Za pohranu podataka aplikacije korištena je relacijska baza podataka PostgreSQL, koja je odabrana zbog svoje pouzdanosti, visoke razine performansi i dobre skalabilnosti u produkcijskim okruženjima. PostgreSQL se odlično integrira sa Spring Boot okvirom te omogućuje sigurno i učinkovito upravljanje podacima. Za

administraciju baze podataka korišten je alat pgAdmin, grafičko sučelje koje omogućuje jednostavno upravljanje bazom, pregled i uređivanje tablica te izvođenje SQL upita tijekom razvoja i testiranja aplikacije.

22.4 Razvojni alati

Za razvoj poslužiteljskog dijela aplikacije korišten je IntelliJ IDEA kao glavni razvojni alat, koji pruža napredne mogućnosti za razvoj, testiranje i debugiranje Java i Spring Boot aplikacija. Klijentski dio aplikacije razvijan je pomoću alata Visual Studio Code, laganog i fleksibilnog razvojnog okruženja prilagođenog radu s JavaScript i React tehnologijama. Za upravljanje verzijama izvornog koda korišten je Git, koji omogućuje praćenje promjena i timski rad na projektu, dok je GitHub korišten kao centralno mjesto za pohranu repozitorija i suradnju među članovima tima. Za pokretanje frontend razvojnog okruženja i upravljanje JavaScript paketima korišten je Node.js, koji je nužan za rad s Vite i React alatima.

22.5 Alati za ispitivanje

Ispitivanje aplikacije provedeno je pomoću alata Selenium WebDriver, koji omogućuje automatizirano testiranje web aplikacija kroz simulaciju ponašanja stvarnog korisnika u web pregledniku. Testovi su razvijani u razvojnem okruženju IntelliJ IDEA, korištenjem programskog jezika Java. Razvojno okruženje IntelliJ IDEA odabrano je jer je isto korišteno i za izradu samog projekta, čime je osigurana konzistentnost razvojnog okruženja i jednostavnije održavanje projekta. Alat Selenium IDE nije korišten jer više nije u potpunosti kompatibilan s aktualnim verzijama preglednika Google Chrome te ne zadovoljava potrebe složenijeg i fleksibilnijeg automatiziranog testiranja. Selenium WebDriver je odabran jer omogućuje pouzdano, fleksibilno i ponovljivo testiranje funkcionalnosti aplikacije, neovisno o korisniku, te jer omogućuje preciznu kontrolu nad ponašanjem preglednika. Za pokretanje testova korišten je odgovarajući WebDriver (ChromeDriver).

22.6 Alati za razmještaj

Za razmještaj aplikacije korištena je platforma Render, koja omogućuje automatsku izgradnju (build) i implementaciju aplikacije iz GitHub repozitorija. Backend i frontend dio aplikacije automatski se izgrađuju i pokreću na Render serverima bez potrebe za ručnom konfiguracijom infrastrukture. Proces razmještaja temelji se na CI/CD principu, pri čemu se svaka promjena u repozitoriju može automatski implementirati u produkcijsko okruženje, čime se pojednostavljuje održavanje i nadogradnja sustava.

22.7 Cloud platforma

Aplikacija je hostana na Render cloud platformi, koja pruža potrebnu infrastrukturu za pokretanje backend i frontend servisa te upravljanje bazom podataka u oblaku. Render omogućuje jednostavno upravljanje aplikacijom, nadzor njezina rada i kontinuiranu dostupnost sustava putem interneta, čime se osigurava stabilno i pouzdano izvođenje aplikacije u produkcijskom okruženju.

23 1. Instalacija

- **Preduvjeti:** Node 22.16.0, Java 21(JDK)
- **Preuzimanje:**
 - `git clone https://github.com/LeProgi/Dvoranko.git`
 - `cd Dvoranko`
- **Instalacija ovisnosti:**
 - `cd ./source_code/frontend/dvoranko-frontend`
 - `npm install`

24 2. Postavke

24.1 Detaljne upute za konfiguraciju aplikacije:

24.1.1 Konfiguracijske datoteke:

Potrebno je definirati varijable za frontend i za backend. Varijable za frontend treba upisati u .env datoteku koju je potrebno stvoriti u mapi Dvoranko/source_code/frontend/dvoranko-frontend. Varijable za backend treba definirati unutar samog text editor-a, npr. putem terminala u VSCode-u. Ako se aplikacija pokreće putem usluge Render, sve varijable su definirane putem sučelja na stranici usluge.

- **Varijable za frontend:**

- VITE_BACKEND_URL(nije potrebna za lokalno pokretanje, URL aplikacije na Render-u)
- VITE_GOOGLE_MAPS_API_KEY

- **Varijable za backend:**

- SPRING_DATASOURCE_URL(URL baze podataka na Render-u, lokalno u formatu jdbc:postgresql://hostname.region-postgres.render.com:port/ime_baze?sslmode=require, na Render-u u formatu jdbc:postgresql://hostname:port/ime_baze?sslmode=require, hostname, region, port i ime_baze treba zamijeniti podacima baze)
- SPRING_DATASOURCE_USERNAME(user baze podataka na Render-u)
- SPRING_DATASOURCE_PASSWORD(password baze podataka na Render-u)
- GOOGLE_OAUTH2_ID
- GOOGLE_OAUTH2_SECRET
- MAIL_USERNAME
- MAIL_PASSWORD
- CLOUDINARY_CLOUD_NAME
- CLOUDINARY_API_KEY
- CLOUDINARY_API_SECRET
- MAILGUN_API_KEY

Za lokalno korištenje aplikacije potrebno je unutar datoteke application.properties u direktoriju Dvoranko/source_code/backend/dvoranko-backend/src/main/resources zamijeniti komentar u linijama(maknuti iz jednog reda i staviti u drugi) u kojima je navedeno “leprogi.dvoranko.url=” i ako se koristi vlastiti web service na Render-u, treba promijeniti URL koji se ne odnosi na Render servis. Također, u datoteci main.jsx u direktoriju Dvoranko/source_code/frontend/dvoranko-frontend/src treba zamijeniti urlPublic u urlLocal u zadnjoj import liniji i urlPublic u URL.js datoteci u istom direktoriju treba se promijeniti ako se koristi vlastiti web service.

24.1.2 Postavke baze podataka:

- Potrebno je napraviti novu bazu podataka na Render-u.
 - Prijavite se na Render.
 - Kreirajte novi **Postgres** servis.
 - Konfigurirajte postavke:
 - * ime postgres instance
 - * ime baze

- * user
- * region, mora biti isti kao region web service-a aplikacije
- * postgresql version: 17
- * instance type

25 3. Pokretanje aplikacije

25.0.1 Upute za pokretanje aplikacije u različitim okruženjima:

- **Razvojno okruženje:**
 - npm run dev
- **Produksijsko okruženje:**
 - Render predstavlja produkcijsko okruženje.
- **Prevođenje aplikacije:**
 - npm run build
- **Pokretanje servera:**
 - U mapi Dvoranko/source_code/backend/dvoranko-backend:
 - * ./mvnw verify
 - * java -jar ./target/dvoranko-0.0.1-SNAPSHOT.jar
- **Provjera rada:**
 - http://localhost:5173

26 4. Upute za administratore

26.0.1 Smjernice za administratore aplikacije nakon puštanja u pogon:

- **Pristup administratorskom sučelju:**
 - prilikom prijave na stranicu s računom koji je admin, biti ćete automatski prebačeni na administratorsko sučelje
- **Redovito održavanje:**
 - **Ažuriranje baze podataka:**

Bazu podataka na Render-u, ako ju se želi održati besplatnom, treba svakih 30 dana ponovno kreirati. Potrebno je napraviti dump stare baze u Command Promptu ili Terminalu pomoću sljedeće komande:

```
* pg_dump "postgresql://user:password@hostname.region-postgres.render.com:port/ime_baze?sslmode=
> ime_datoteke.sql(zamijeniti user, password, host, region, port, ime_baze i ime_datoteke
s podacima baze i željene datoteke)
```

Nakon toga treba pratiti upute navedene ranije za stvoriti novu bazu i kada ju se kreira treba ju očistiti za slučaj da se u njoj nalaze neželjeni podatci. To se može napraviti povezivanjem na bazu putem Command Prompta ili Terminala s komandom:

```
* psql "postgresql://user:password@hostname.region-postgres.render.com:port/ime_baze?sslmode=
```

i izvršavanjem sljedeće 3 komande:

```
* DROP SCHEMA public CASCADE;
* CREATE SCHEMA public;
* GRANT ALL ON SCHEMA public TO postgresuser;
```

Nakon toga treba se ugasiti veza s bazom pomoću “\q” i uvesti stari dump u novu bazu pomoću sljedeće komande:

```
* psql "postgres://user:password@hostname.region-postgres.render.com:port/ime_baze?sslmode=
-f ime_datoteke.sql
```

Zastarjele podatke treba arhivirati pa izbrisati iz baze.

- Pregled logova, objašnjeno kasnije.
- Aplikacija se ažurira tako da se povuče nova verzija iz Git repozitorija i ponovno pokrene frontend i server.

```
* git pull origin main
* npm install, u ranije navedenom frontend direktoriju
* npm run dev, također u frontend direktoriju
* pokretanje servera, navedeno ranije
```

Na Render-u se aplikacija sama ažurira pri svakom commit-u na branch na koji je povezan servis.

- **Rješavanje problema:**

- Na Render-u se unutar servisa aplikacije na lijevoj strani ekrana mogu pronaći “Logs” i tamo se mogu pročitati log-ovi za svako pokretanje aplikacije.

27 5. Postavljanje na platformu Render

Render je popularna cloud platforma za jednostavno smještanje aplikacija. Ova aplikacija napravljena je s tom platformom pa se nju preporuča koristiti.

- **Priprema repozitorija:**

- Osigurajte da vaš projekt ima datoteku Dockerfile za konfiguraciju deploy-a. Datoteka je dostupna na GitHub-u i kopira se prilikom kopiranja repozitorija.

- **Postavljanje na Render:**

- Prijavite se na Render.
- Kreirajte novi **Web Service** i povežite ga s vašim GitHub repozitorijem.
- Konfigurirajte postavke:
 - * ime aplikacije
 - * jezik: Docker
 - * branch s kojeg se povlače podatci
 - * region, mora biti isti kao region baze podataka
 - * root directory: .
 - * instance type
 - * varijable, navedene ranije

* Dockerfile Path: ./Dockerfile(to je default)

- **Pokretanje aplikacije:**

- Render će automatski preuzeti repozitorij, instalirati ovisnosti i pokrenuti aplikaciju. Nakon deploymenta, aplikaciji možete pristupiti putem generiranog URL-a, <https://ime-aplikacije.onrender.com>.

28 6. Opis pristupa aplikaciji na javnom poslužitelju

Za pristup aplikaciji putem web poslužitelja, koristite URL, <https://dvoranko.onrender.com>.

Ako, nakon logout-a, prilikom sljedećeg login-a ne dobijete opciju za biranje računa, nego Vam se automatski izabere zadnje korišten račun, pokušajte koristiti neki drugi browser. Preporučamo Chrome.

Za pristup administratorskom sučelju treba biti ulogiran s računom koji ima admin ulogu. Onda ćete biti prebačeni na sučelje i nećete imati pristup ostatku stranice. Projekt je rađen u timu od šestoro članova. Uz jednog voditelja, ostale uloge bile su vezane uz područje razvoja, odnosno podijeljene na timove koji su se bavili backendom, frontendom te deploymentom. Cilj projekta bio je razvoj web stranice za rezervaciju dvorana na području Zagreba. Svrha tih dvorana može biti raznolika, od sportskih aktivnosti do kulturnih događanja i slično.

Tijekom razvoja stranice prepoznali smo određene tehničke izazove:

- Validacija korisničkog unosa – odobravanje i provjera unosa prilikom objave i rezerviranja dvorane
- Responzivni dizajn – prilagodba koda kako bi stranica bila prikladna za različite veličine ekrana
- Implementacija sustava rezervacija
- Korištenje Maps API-ja
- Testiranje funkcionalnosti kroz sve predviđene use-caseove
- Deployment stranice na uslugu Render

Smatramo kako smo sve izazove na koje smo naišli uspjeli izravno riješiti, a u slučajevima gdje to nije bilo moguće, pronašli smo odgovarajuća privremena rješenja (work-around).

Tijekom izrade projekta stečena su brojna nova znanja. U području backend razvoja stečeno je iskustvo rada sa Spring Boot okvirom, uključujući povezivanje web stranice s bazom podataka, obradu korisničkih zahtjeva, implementaciju logike backend dijela aplikacije te rad s HTTP zahtjevima.

U području frontend razvoja stečena su znanja rada s React bibliotekama, izrada komponenti, komunikacija s backendom putem API poziva te izrada responzivnog korisničkog sučelja. Također su stečena dodatna znanja o osnovnim načelima dizajna web stranica.

Korištenjem Maps API-ja stečeno je iskustvo integracije vanjskih servisa u aplikaciju. Također je stečeno znanje korištenja Cloudinary servisa za pohranu i upravljanje slikama unutar web stranice.

Stečena su i znanja vezana uz postavljanje stranice na uslugu Render i način na koji se tamo frontend i backend pokreću i povezuju. Ovim putem naučili smo se koristiti .yaml datotekama, koje nam na kraju nisu bile potrebne i Dockerfile datotekama.

Jedno od najkorisnijih znanja koje smo stekli dok smo radili projekt je korištenje s GitHub-om i Git-om.

Za brži i kvalitetniji razvoj projekta bila bi potrebna naprednija znanja iz gore navedenih područja, kao i bolje razumijevanje organizacije timskog rada te testiranja funkcionalnosti web stranice. Dodatno iskustvo u radu s vanjskim API-jima omogućilo bi još kvalitetniju implementaciju funkcionalnosti i bolji vizualni dojam stranice.

Perspektive daljnjeg rada u projektnoj grupi uključuju poboljšanje postojećeg sustava rezervacija, proširenje aplikacije na druge gradove kroz mogućnost odabira područja za pregledavanje dvorana te uključivanje drugih tipova prostora, poput kazališta, koncertnih dvorana i konferencijskih sala. Daljnji razvoj mogao bi

obuhvaćati dodatnu optimizaciju funkcionalnosti, unapređenje vizualnog izgleda stranice, uvođenje online sustava automatskog plaćanja rezervacija, razvoj mobilne aplikacije za Android i iOS platforme, implementaciju sustava ocjenjivanja prostora te prikaz statistike vlasnicima prostora, uključujući broj novih klijenata i ukupan broj rezervacija. Time bi se dodatno poboljšalo korisničko iskustvo, povećala transparentnost među korisnicima i omogućilo učinkovitije upravljanje prostorima. Kontinuirano osvježavanje Popisati sve reference i literaturu koja je pomogla pri ostvarivanju projekta.

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. I. Sommerville, "Software engineering", 8th ed, Addison Wesley, 2007.
3. T.C.Lethbridge, R.Langaniere, "Object-Oriented Software Engineering", 2nd ed. McGraw-Hill, 2005.
4. I. Marsic, Software engineering book, Department of Electrical and Computer Engineering, Rutgers University, <http://www.ece.rutgers.edu/~marsic/books/SE>
5. The Unified Modeling Language, <https://www.uml-diagrams.org/>
6. Astah Community, <http://astah.net/editions/uml-new>

Rev. | Opis promjene | Autor | Datum | | | | | | | | | | 1.1. | Napravljen predložak za wiki | D.Maras | 13.10.2025. | | 1.2. | Informacije o timu | E.Huljaj | 13.10.2025. | | 1.3. | Početni opis projektnog zadatka | E.Huljaj | 23.10.2025. | | 1.4. | Raspisani funkcionalni i nefunkcionalni zahtjevi | E.Huljaj | 25.10.2025. | | 1.5. | Opis tablica baze podataka | I.Vučak | 29.10.2025. | | 1.6. | Opis arhitekture sustava | I.Vučak | 29.10.2025. | | 1.7. | Dijagram baze podataka | I.Vučak | 29.10.2025. | | 1.8. | Opis dionika sustava | E.Huljaj | 1.11.2025. | | 1.9. | Nadopunjen opis baze podataka (popravljen tablice i dijagrami) | I.Vučak | 3.11.2025. | | 1.10. | Dijagrami obrazaca uporabe | D.Maras | 3.11.2025. | | 1.11. | Interakcije glavnih korisničkih aktera sa sustavom | D.Maras | 3.11.2025. | | 1.12. | Opis obrazaca uporabe | D.Maras | 3.11.2025. | | 1.13. | Ispravak obrazaca uporabe | E.Huljaj | 10.11.2025. | | 1.14. | Ispravak funkcionalnih zahtjeva prema obrascima uporabe | E.Huljaj | 10.11.2025. | | 1.15. | Provjera uključenosti funkcionalnosti u obrasce uporabe | E.Huljaj | 10.11.2025. | | 1.16. | Ispravak opisa projektnog zadatka prema novim F/NF zahtjevima | E.Huljaj | 11.11.2025. | | 1.17. | Napisan opseg projektnog zadatka | E.Huljaj | 11.11.2025. | | 1.18. | Opisani frontend i backend slojevi aplikacije, popravljen opis arhitekture | M.Vandek | 12.11.2025. | | 1.19. | Sekvencijski dijagrami | D.Maras | 12.11.2025. | | 1.20. | Popravljen nazivlje u tablicama baze podataka | I.Vučak | 13.11.2025. | | 1.21. | Ispravak popisa dionika | V.M.Lulić | 13.11.2025. | | 1.22. | Dodani opisi svih slika dijagrama | D.Maras | 13.11.2025. | | 1.23. | Dijagram razreda (slike i opis dijagrama) | D.Maras | 13.11.2025. | | 2.1. | Tehnologije za implementaciju | E.Huljaj | 19.1.2026. | | 2.2. | Dijagrami stanja i aktivnosti | D.Maras | 21.1.2026. | | 2.3. | Dijagram komponenti i dijagram razmještaja | D.Maras | 21.1.2026. | | 2.4. | Ispitivanje komponenti | M.Vandek | 21.1.2026. | | 2.5. | Upute za puštanje u pogon | I.Vučak | 21.1.2026. | | 2.6. | Zaključak i budući rad | B.Navratil | 21.1.2026. | | 2.7. | Arhitektura i dizajn sustava | I.Vučak | 22.1.2026. | | 2.8. | Zaključak i budući rad | I.Vučak | 22.1.2026. | | 2.9. | Specifikacija zahtjeva sustava | I.Vučak | 22.1.2026. | | 2.10. | Slike korisničkog sučelja | E.Huljaj | 22.1.2026. | | 2.11. | Opis tehnologija za ispitivanje | E.Huljaj | 23.1.2026. | | 2.12. | Ispitivanje sustava | V.M.Lulić | 23.1.2026. |

*Rev. -> (ciklus).(broj promjene)

29 Dnevnik sastajanja

U ovom dijelu potrebno je redovito osvježavati dnevnik sastajanja/tjedni izvještaj aktivnosti prema predlošku. Cilj predstaviti kratki opis ključnih sastanaka, **donesenih odluka** i zaduženja s tjednih sastanaka.

1. sastanak

- Datum: 9. listopada 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, M.Vandek, D.Maras, I.Vučak, E.Huljaj
- Teme sastanka: > * uspostavljena Discord grupa sa svim članovima tima > * odabran naziv tima > * diskusija o vlastitoj ideji kao temi projekta te sastavljanje prijedloga > * inicijalna i okvirna podjela članova tima po ulogama

2. sastanak

- Datum: 16. listopada 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, I.Vučak, E.Huljaj
- Teme sastanka: > * određeni funkcionalni i nefunkcionalni zahtjevi > * daljnja razrada funkcionalnosti aplikacije > * razrada strukture baze podataka

3. sastanak

- Datum: 3. studenoga 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, I.Vučak, E.Huljaj, D.Maras, M.Vandek
- Teme sastanka: > * definirani obrasci uporabe > * definiran način implementacije Google Maps API funkcionalnosti > * određene potrebne promjene u izgledu web-aplikacije » * kako pristupamo login stranici » * popis dvorana prebaciti na početnu stranicu

4. sastanak

- Datum: 12. prosinca 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, I.Vučak, E.Huljaj, M.Vandek, D.Maras
- Teme sastanka: > * definiran izgled i funkcionalnosti admin i renter(iznajmljivač) stranica > * dogovorena logika i izgled zahtjeva za kreiranje nove dvorane > * određen način provjere i spremanja podataka adrese pri objavi nove dvorane

5. sastanak

- Datum: 19. prosinca 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, I.Vučak, E.Huljaj, M.Vandek, D.Maras
- Teme sastanka: > * provjera do sad napravljenih funkcionalnosti dogovorenih na prošlom sastanku > * dogovoren finalni izgled ProfilePage-a ovisno o role-u korisnika > * određen način brisanja korisnika i dvorana > * diskusija o daljnjem planu

6. sastanak

- Datum: 11. siječnja 2026.
- Prisustvovali: V.M.Lulić, I.Vučak, E.Huljaj, M.Vandek, D.Maras
- Teme sastanka: > * dogovoren način integracije mail servisa > * osmišljen način rezervacije termina uz korištenje Google kalendara > * pregledane sve funkcionalnosti te su podijeljene one koje još nisu završene

7. sastanak

- Datum: 18. siječnja 2026.
- Prisustvovali: V.M.Lulić, E.Huljaj, B.Navratil D.Maras
- Teme sastanka: > * provjera preostalih dorada > * podijeljena izrada dokumentacije > * diskutirane nedoumice oko pisanja dokumentacije > * dogovoreni završni rokovi prije druge predaje

30 Plan rada

Gantt prikaz

31 Tablica aktivnosti

Kontinuirano osvježavanje

Komponente	Vid Martin Lulić	Eva Huljaj	Dino Maras	Ivan Vučak	Matija Vandek	Borna Navratil
Upravljanje projektom	10					
Opis projektnog zadatka		6				
Funkcionalni zahtjevi	1	4.5		1.5	1	2
Opis pojedinih obrazaca		1	5			
Dijagram obrazaca			2.5			
Sekvencijski dijagrami			3.5			
Opis ostalih zahtjeva		1				

Komponente	Vid Martin Lulić	Eva Huljaj	Dino Maras	Ivan Vučak	Matija Vandek	Borna Navratil
Arhitektura i dizajn sustava	2			8		
Izrada baze podataka				5	3	
Spajanje backenda s bazom podataka	3		1			
Dijagram razreda			2.5			
Dijagram stanja			5			
Dijagram aktivnosti			2			
Dijagram komponenti			2			
Učenje tehnologija i alata	20	5	10		5	5
Ispitivanje programskog rješenja	12				8	
Dijagram razmještaja			3			
Upute za puštanje u pogon				8		
Sastanci	10	10	10	10	10	10
Zaključak i budući rad				1		3
Popis literature						
Uređivanje dokumentacije	1	3	1	2	3	
Izrada korisničkog sučelja	20	35	6	46.5	25.5	30
Izrada programske potpore	42		38	1.5	11.5	
Izrada prezentacije		2				
Dizajn korisničkog sučelja		6				5
Pregledavanje dokumentacije	2	2	2		2	
Puštanje aplikacije u pogon	5			18		

32 Dijagram pregleda promjena

Dijagram pregleda promjena nad datotekama projekta, prikaz commitova tijekom trajanja projekta.

33 Kjučni izazovi i rješenja

Tijekom izrade projekta tim se susreo s nekoliko izazova, među kojima su se istaknuli provođenje ispitivanja aplikacije, učenje novih programskih biblioteka i alata te izrada dokumentacije, s obzirom na to da u početku nije bilo jasno kako je strukturirati i pravilno izraditi. Dodatno vrijeme bilo je potrebno za istraživanje, rješavanje tehničkih problema i konfiguraciju alata za testiranje, što je povremeno usporavalo napredak projekta. Unatoč tome, izazovi su uspješno savladani kroz suradnju članova tima i postupno usvajanje novih znanja. Kroz rad na projektu stečeno je vrijedno iskustvo u automatiziranom testiranju, organizaciji rada i izradi tehničke dokumentacije, što će biti korisno u budućem radu.

TG10.1 LeProgi

Dvoranko

Voditelj: Vid Martin Lulić

Tim: D. Maras, V. Vučić, E. Hujak, B. Navratil, M. Vrandeć

Project start: ned, 10.5.2025

Display week: 1

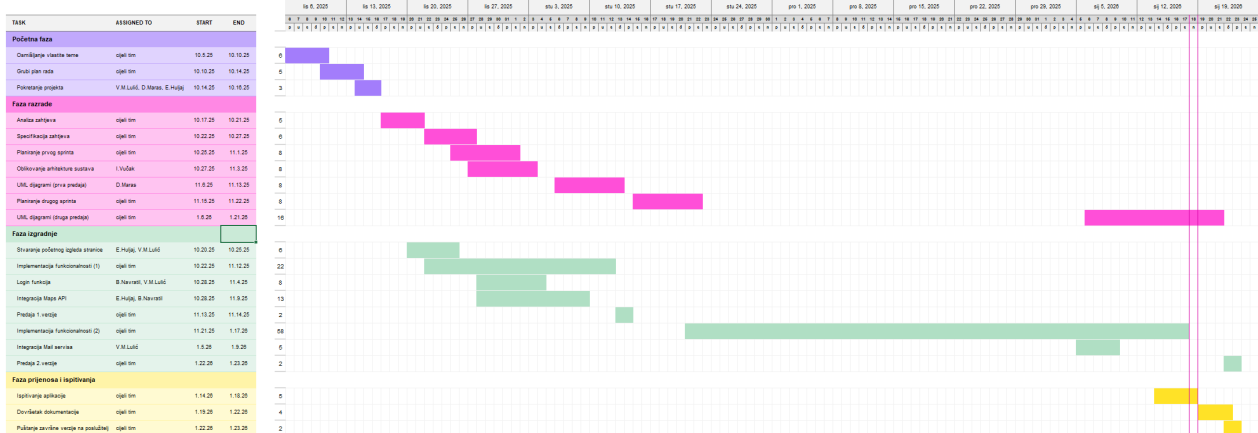


Figure 27: Slika

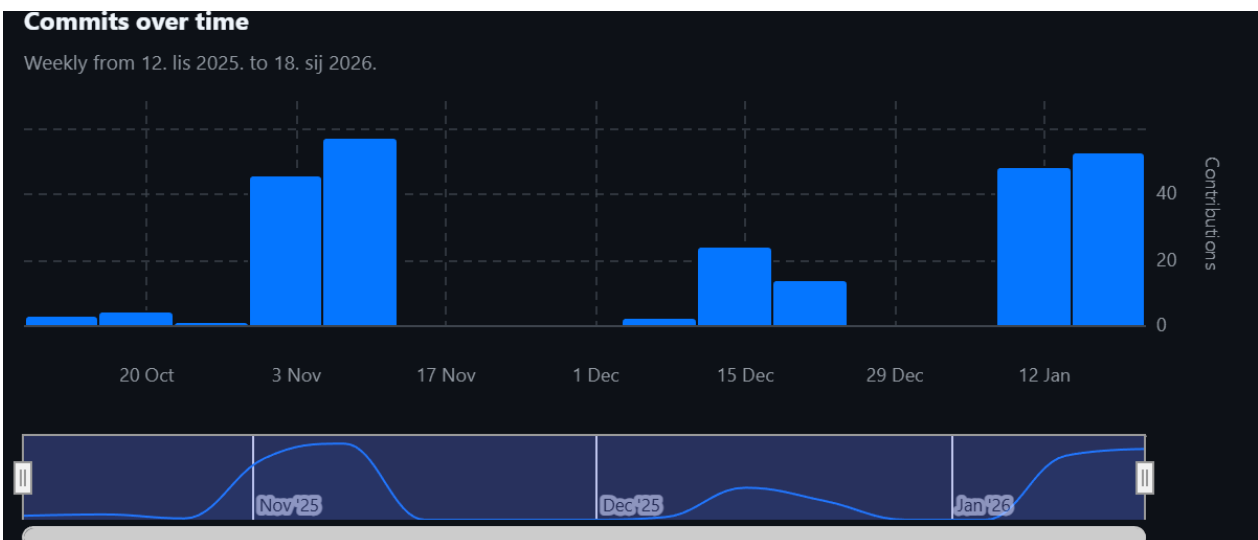


Figure 28: Slika